

Draw.io

A Nasdanika Zephyr Book

Pavel Vlasov

Version 2026.11.0, 2026-11-07

Table of Contents

Colophon . . . 1

Preface . . . 3

Who this book is for . . . 3

A book that stands on its own . . . 3

What you need . . . 4

Conventions . . . 4

I: The Why . . . 6

1. Diagrams are communication . . . 7

1.1. What a diagram is . . . 7

1.2. Who a diagram is for . . . 8

1.3. The cost of a diagram . . . 8

1.4. What I wanted from a diagram . . . 9

2. The visual communication continuum . . . 10

2.1. Picture . . . 10

2.2. Interactive diagram . . . 11

2.3. Documented diagram . . . 11

2.4. Diagram as data . . . 11

2.5. Executable diagram . . . 12

2.6. Diagram as a model . . . 12

2.7. Choosing a point . . . 13

3. What a diagram is made of . . . 14

3.1. Nodes, connections and what they stand for . . . 14

3.2. Visual features . . . 15

4. Why Draw.io . . . 17

4.1. What it gives you . . . 17

4.2. What it does not give you . . . 18

4.3. The file format . . . 18

4.4. Diagrams in PNG images . . . 18

4.5. The module . . . 19

II: The How . . . 20

5. Getting started . . . 21

5.1. Adding the module . . . 21

5.2. The running example . . . 22

5.3. Loading a diagram . . . 23

5.4. Saving and loading . . . 24
5.5. Sharing a diagram . . . 25
6. The document model . . . 26
6.1. From document to element . . . 26
6.2. The figure checks the API . . . 27
6.3. Types and generics . . . 28
7. Navigating a diagram . . . 30
7.1. Walking the containment . . . 30
7.2. Streams . . . 30
7.3. Connections . . . 32
7.4. Visitors . . . 32
7.5. Connection base . . . 33
7.6. Tags and layers . . . 34
8. Properties and placeholders . . . 35
8.1. Reading properties . . . 35
8.2. Placeholders . . . 35
8.3. Properties from outside the diagram . . . 36
8.4. Filtering properties . . . 37
8.5. Writing properties . . . 38
9. Selectors and property paths . . . 39
9.1. Segments . . . 39
9.2. Following connections . . . 39
9.3. Containment . . . 40
9.4. Across pages . . . 40
9.5. Property paths . . . 40
9.6. When nothing matches . . . 41
10. Links between pages, elements and documents . . . 43
10.1. Link syntax . . . 43
10.2. Resolving links . . . 44
10.3. Following links . . . 44
10.4. Other documents . . . 45
10.5. Logical URIs . . . 46
10.6. Element URIs . . . 47
11. Creating and changing diagrams . . . 48
11.1. Creating a diagram . . . 48
11.2. Styles . . . 49

11.3. Connections . . . 49
11.4. Saving and reading back . . . 50
11.5. Changing a diagram that people drew . . . 51
11.6. Removing elements . . . 51
12. Geometry and order . . . **53**
12.1. Orders that are not geometry . . . 53
12.2. Geometry . . . 54
12.3. Enumeration . . . 55
12.4. Comparators that are not orders . . . 55
12.5. The continuum, again . . . 55
13. Rendering in a browser . . . **57**
13.1. Why draw.io's own viewer . . . 57
13.2. The module . . . 57
13.3. A page as PNG . . . 58
13.4. A page as SVG . . . 59
13.5. Several pages . . . 60
13.6. Interacting before the picture . . . 60
13.7. Videos . . . 62
13.8. Rendering options . . . 62
13.9. Limits . . . 63
III: Looking Ahead . . . **64**
14. The rest of the continuum . . . **65**
14.1. Documented diagrams . . . 65
14.2. Executable diagrams . . . 65
14.3. Diagrams as models . . . 66
14.4. One diagram model, many pictures . . . 66
14.5. Rendering generated diagrams . . . 67
14.6. Diagrams and AI assistants . . . 67
Appendix A: API reference . . . **68**
A.1. `org.nasdanika.sdk.runtime.drawio` . . . 68
A.2. `org.nasdanika.sdk.runtime.drawio.style` . . . 69
A.3. `org.nasdanika.sdk.runtime.drawio.comparators` . . . 69
A.4. `org.nasdanika.sdk.runtime.drawio.util` . . . 70
A.5. `org.nasdanika.sdk.runtime.drawio.playwright` . . . 71
A.6. Selectors . . . 71
A.7. Links . . . 71

Appendix B: Migrating from Nasdanika core **73**
B.1. What moves 73
B.2. What was left out 73
B.3. Dependencies 74
Appendix C: Building this book, and editing its figures **75**
C.1. Building 75
C.2. Conventions 75
C.3. Editing the figures 76
Appendix D: Glossary **77**

Colophon

Draw.io: A Nasdanika Zephyr Book

Edition 2026.11.0, built on 2026-11-07.

This edition describes the `drawio` module of the Nasdanika SDK Runtime 2026.11.0, Maven coordinates `org.nasdanika.sdk.runtime:drawio:2026.11.0`. Every code listing in it is included from the module's tests, and every figure is a PNG image with its Draw.io diagram embedded, which those tests load. When the book is built, the tests run first, so a listing in this book is code that compiled and passed against this version of the module, and a figure shows the very diagram its test read.

Copyright © 2026 Pavel Vlasov.

The text and figures of this book are licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0). You may share and adapt them for any purpose, commercial purposes included, provided you give appropriate credit, provide a link to the license, and indicate if you made changes. A suggested credit: "Pavel Vlasov, *Draw.io: A Nasdanika Zephyr Book*, licensed under CC BY 4.0", with a link to github.com/Nasdanika-SDK/runtime.

The code is licensed under the Eclipse Public License 2.0, the license of the runtime. In addition, you may use the code of this book's listings in your own programs and documentation without asking for permission and without attribution.

Neither license covers the Nasdanika name or logo: CC BY 4.0 grants no trademark rights.

draw.io and diagrams.net are trademarks of JGraph Ltd. Java and OpenJDK are trademarks or registered trademarks of Oracle and its affiliates. Eclipse and EMF are trademarks of the Eclipse Foundation. Other names may be trademarks of their owners. Their use here identifies the technologies the module works with and implies no endorsement. This book and the module are not affiliated with JGraph Ltd.

The figures were drawn in Draw.io and rendered by Draw.io's own viewer. The source of this book, its tests and the module it describes are at github.com/Nasdanika-SDK/runtime, in the `drawio` directory.

Typeset with Asciidoctor PDF in Noto Sans and M+ 1mn, in the look of the Nasdanika Zephyr Books.

Preface

This book is about diagrams that people draw and programs read. Most diagrams in an organization are drawn once, looked at a few times and then left to drift away from what they describe. They do not have to be. A diagram drawn in Draw.io is a structured document, and with a little discipline it can be documentation, configuration, a program's wiring or the source of a model, while it stays a picture that anyone can open and change.

The first part of the book is the why: what diagrams are for, how much structure a particular diagram should carry, and why Draw.io is a good editor to build on. The second part is the how, for Java developers: the Draw.io module of the Nasdanika SDK Runtime, which reads, navigates, changes and writes Draw.io documents. The third part says what is coming next.

Who this book is for

People who draw. Architects, analysts and engineers who already use Draw.io, and who would like their diagrams to do more than sit in a wiki. Part one is for you, and needs no code.

Java developers who are handed diagrams and asked to do something with them: check them, generate from them, publish them, or keep them in step with a system. Read the book in order; part two assumes part one.

Anyone who builds tools on diagrams. The module is small and has few dependencies, and its document model mirrors Draw.io's file format closely enough that what you learn here holds for the format itself.

A book that stands on its own

This book is one of the Nasdanika Zephyr Books: in-depth technical manuals, one per module of the Nasdanika SDK Runtime, or per small group of modules used together, in the manner of IBM Redbooks. It covers two: `drawio`, the API, and `drawio-playwright`, which renders diagrams in a browser. The runtime has a book of its own, *Nasdanika SDK Runtime: More Structure with Less Effort*, which is the overview: why the runtime exists, its principles and its modules; its chapter "Diagrams" introduces these modules among the runtime's other diagram modules. You do not need to read it first. Where the two books touch, this one repeats what you need, so that someone who has never heard of the runtime and only wants to read their diagrams from Java

can start and finish here. Where the runtime goes further than this module, the last part says so and points to it.

The series is named after the Bootswatch theme it is styled with, Zephyr. The books have nothing to do with the Zephyr real-time operating system of the Linux Foundation, or with SmartBear's Zephyr test management.

The why in this book grew out of two earlier pieces of mine: the book *Beyond Diagrams* (2024), written for the Draw.io API in Nasdanika core, from which this module was salvaged, and the Medium story "Visual communication continuum" (medium.com/nasdanika). This book keeps their argument and rebuilds their how on the runtime.

What you need

- **A JDK for Java 25** and **Maven 3.9.16 or later** to build against the module. Its Java module is `org.nasdanika.sdk.runtime.drawio`, and its Maven coordinates are `org.nasdanika.sdk.runtime:drawio:2026.11.0`.
- **Draw.io**, to draw: the desktop application, the web application at www.drawio.com, or the Draw.io extension for Visual Studio Code. All of them read and write the same files.
- **Chrome, Edge or Playwright's Chromium**, only to render diagrams as images or videos (Chapter 13).

Conventions

- **Every listing is included from a test** of the module and compiled and run when the book is built. Callouts, such as <1>, explain the lines they mark.
- **Every figure is a diagram.** The figures are PNG images with the Draw.io diagram embedded. Open one in Draw.io and you can edit it; load it with the API and you get the document the listings work on (Chapter 5).
- **Status labels** mark what this module does not do yet: PLANNED for designed but not built, SKETCH for partly built.
- **Voice.** I write in the first person. "The author" is ten times as many letters, and a book about effective communication should not start with an ineffective construct.



Editor's notes like this one record inconsistencies between the module and its

> documentation found while writing. They are shown in builds for the author and left out of published editions (`-Dzephyr.editor-notes=false`, set by the `central` profile).
>
> The continuum figure (Chapter 2) still draws "Diagram as a model" as planned. Since 2026-10-25 the runtime's Draw.io model loads diagrams into resource sets as models and the diagram model holds semantic elements; only mapping by properties is planned. Redraw the point as partly built when the figure is next edited in Draw.io.

Part I: The Why

What diagrams are for, how much structure a diagram should carry, and why Draw.io is the editor this book builds on. No code is needed to read this part.

Chapter 1. Diagrams are communication

Diagrams are older than writing. People drew maps, tallies and kinship long before they wrote sentences, and they still reach for a whiteboard when a conversation gets complicated. You might expect such an old form of communication to be well understood. In my experience it is not: there is a gap in how people approach diagramming, and it costs organizations a great deal of time.

Like writing, diagramming is not an innate skill. Being able to connect two boxes with an arrow does not make someone a good diagrammer, any more than being able to write a sentence makes someone a good writer. And I've seen many diagrams where people can't even connect two boxes correctly - they draw arrows between boxes, but the arrows do not connect to the boxes - so there is no relationship. This part of the book is about thinking in diagrams: what a diagram is for, who it is for, and how much structure it should carry.

1.1. What a diagram is

A diagram is a symbolic representation of information, and in the sense this book uses, a particular genre of it: shapes connected by lines, arrows or other visual links, which show things and their relationships. I extend the genre to quantitative information too, because the size, position and color of a shape can carry numbers as well as categories (Chapter 3).

Diagrams fall into two broad groups.

Representations only. The diagram leaves it to the beholder to work out what a shape means. A blue rounded rectangle might be a system, a team or a deployment unit, and only a legend, a convention or a conversation says which. Draw.io, Visio and PowerPoint produce diagrams of this kind.

Representations in a domain language. UML, BPMN, ArchiMate and SysML define what each shape means, so a reader who knows the language reads the diagram without help.

Enterprise IT is dominated by the first kind, and Draw.io and PowerPoint are its common formats. UML has been around for about thirty years and is still used narrowly and often misused. Simon Brown, the author of the C4 model, made the same observation: many teams have abandoned the modeling languages for simpler boxes-and-lines diagrams, and in doing so

have lost some of their ability to communicate visually.

I think the reason is that every enterprise has a language of its own, rarely written down. It speaks of products, channels, platforms, capabilities, controls and owners, with meanings that are local to it. It does not speak UML or BPMN, and a tool that imposes either one is resisted. Tools that make it easy to build a specialized diagram editor for such a language, Eclipse Sirius among them, are not widely used either, because a specialized editor is one more tool to install, learn and keep.

What remains is the general-purpose editor and the boxes-and-lines diagram, which everybody can draw and nobody can check.

1.2. Who a diagram is for

A diagram is a message, and a message has readers. I find it useful to name four of them.

Yourself, now. You draw to understand the problem in front of you and to break it into pieces. Such a diagram may never be shown to anyone, and that is fine.

Yourself, later. The faintest ink is better than the strongest memory. What is obvious today will not be obvious in a year, when you come back to a decision and need to know why it was made.

Other people. You draw to explain an intent, to gather feedback, or to instruct. This is where most diagrams are aimed, and where most of their cost lies: every time a reader has to ask the author what a shape means, the diagram has failed a little.

Computers. You draw to tell a program what to do: to configure it, to generate code or documentation from the diagram, or to execute it. And, increasingly, **AI assistants**, which read diagrams as context, summarize them for people, or let people ask questions of them. An assistant is a reader that does not get tired of asking what a shape means, as long as the diagram has an answer.

1.3. The cost of a diagram

A diagram has two costs. The **production cost** is the effort of drawing it and keeping it current. The **consumption cost** is the effort its readers spend to get what they need from it: to find it, to understand it, to ask about what they do not understand, and to act on it.

I call a diagram **efficient** when it is cheap to produce and **effective** when it is cheap to consume. The two pull against each other. A sketch is cheap to draw and costly to read, because everything not on the page has to be explained in person. A diagram with documented elements, links between levels of detail and properties that programs can check is more work to draw and much less work to read.

The right balance depends on the readers. A whiteboard sketch for a one-hour meeting should stay a sketch. An architecture diagram that a hundred people consult over three years deserves documentation, and one that drives a deployment deserves properties that a program can check. The goal is the least **total** effort to communicate, production and consumption together.

1.4. What I wanted from a diagram

For years I wanted a way of working with diagrams that would let me:

- create a diagram in an editor that everyone can use;
- share it with people who do not have that editor, in a browser;
- document its elements with rich text, beyond what a tooltip holds;
- give its elements sub-diagrams, nested as deeply as needed;
- read it from a program, as data;
- associate behavior with its elements, so that the diagram is executable;
- and map its elements to a model with a defined meaning, when a defined meaning is what the readers need.

Each of these is a point on a continuum, from a bare picture to a model. The next chapter walks along it.

Chapter 2. The visual communication continuum

A diagram does not have to be either a picture or a model. Between the two lies a continuum of small steps, and each step adds a little structure to the same file. A diagram can stop at any point and move further later, when its readers need more, without being redrawn in another tool.

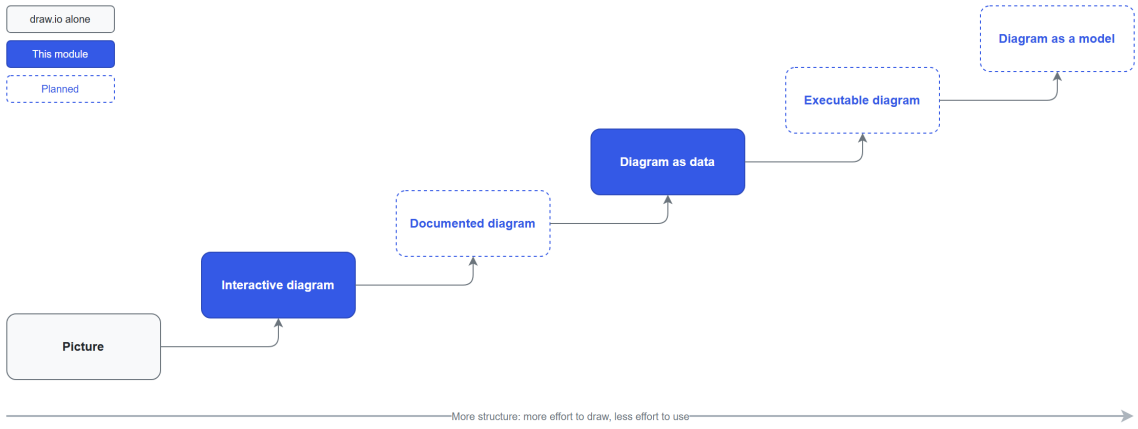


Figure 1. The visual communication continuum. Points in solid blue are what this module does; dashed points are planned.

The figure is itself a diagram that a program reads. Its points carry a `status` property, and the arrows between them say what each step adds. Chapter 12 loads it and puts the points in the order of the arrows, which is how this chapter could be generated from its figure rather than written next to it.

2.1. Picture

Shapes, lines and labels. The meaning is in the heads of the people who drew it and of those who were in the room.

A picture is the right place to stop more often than people think. It is cheap to draw, it gets the problem out of one head and onto a page, and it is often thrown away after the meeting it was drawn for. Its consumption cost is high, but if it has few readers, that cost is never paid.

2.2. Interactive diagram

The same picture, with a little more of the author's knowledge put into it:

- **tooltips** on elements, so a reader can hover to see what a shape is;
- **links** from an element to another page of the diagram, for the next level of detail, or to a page elsewhere;
- **layers** and **tags**, so that one diagram shows several levels of detail, or several concerns, and a reader or a presenter turns them on and off;
- a **viewer in the browser**, so that anyone can read the diagram, click through its pages and toggle its layers without a Draw.io installation.

All of this is plain Draw.io. What the module adds at this point is the ability to do it from code: to publish a diagram as a page with the viewer embedded (Chapter 5), and to check that its links resolve (Chapter 10). **AVAILABLE**

2.3. Documented diagram

Elements get documentation: a paragraph, a page, a page with sub-diagrams. A documentation site is generated from the diagram, with a page per documented element, a navigation tree that follows the containment and the links of the diagram, and search over the text of the pages and of the diagrams themselves.

This is the first point where the consumption cost drops sharply. A reader no longer has to ask the author what an element is; it has a page. The author documents what matters now and leaves the rest for later, for others, or for never. An element without documentation still appears on the diagram; it simply has no page.

Nasdanika core generates such sites from Draw.io diagrams today, and the Internet Banking System demo is one. In the runtime, site generation is **PLANNED** (Chapter 14).

2.4. Diagram as data

Elements get **properties** with agreed meanings: an `owner`, a `type`, a `protocol`, a `criticality`, a `cost-center`. Draw.io stores them with the element and shows them in its *Edit Data* dialog. A program can then read the diagram as data:

- to **check** it: every system has an owner, no customer-facing system talks to the mainframe directly, every connection has a protocol;
- to **report** from it: the systems each team owns, the connections that cross a trust boundary;
- to **configure** something with it: a diagram of a deployment, a pipeline or an integration, from which a program reads its settings;
- to **change** it: color the systems that are past their retirement date, add a tag to every element a change touches, and save the diagram for people to see.

This is the point the module is built for, and most of part two is about it. **AVAILABLE**

2.5. Executable diagram

Behavior is bound to elements. A property names the code that an element stands for, a processor is created for each element, and processors talk to each other along the connections of the diagram. The diagram is the program's wiring, and someone who can draw can rewire it.

Nasdanika core has a framework for executable graphs and diagrams, with processors, handlers and endpoints wired along connections, synchronously or concurrently. In the runtime, it is **PLANNED**.

2.6. Diagram as a model

Elements are mapped to the objects of a typed model, with a metamodel that defines what a system, a team or a capability is, which properties each has, and how they relate. The diagram becomes one of the sources of that model. Everything a model gives is then available: validation against the metamodel, references across diagrams, spreadsheets and code, generation, and tools that understand the model without knowing that it was drawn.

This is the point where the enterprise's own language gets written down, as a metamodel, while the people who speak it keep drawing in the editor they know. In the runtime, the first half of this point is built, outside this module: the runtime's Draw.io model loads a diagram into a resource set as a model, and its elements, as elements of the runtime's diagram model, can depict the objects of any other model (Chapter 14). Mapping them by properties alone, without code, is **PLANNED**.

2.7. Choosing a point

Every step to the right costs the author more and the readers less. A few rules of thumb have served me well:

- **Start on the left.** Draw the picture first. Structure added before anyone has looked at the diagram is often structure nobody needed.
- **Move right when readers ask the same question twice.** The second "what is this?" is a tooltip, the second "where are the details?" is a link or a page, and the second "who owns this?" is a property.
- **Move right when a program needs the answer.** As soon as a check, a report or a deployment depends on what the diagram says, the diagram is data, whether or not anyone admits it. Better to admit it and give its properties names.
- **Never redraw to move.** Each point keeps the file of the point before it. A documented diagram is still a picture, and a diagram mapped to a model can still be opened, read and changed by someone who has never heard of the model.

The rest of this book is about the middle of the continuum: diagrams as data, read and written from Java.

Chapter 3. What a diagram is made of

Before Draw.io, a look at diagrams in general: what they are made of, whatever the tool, paper and pencil included. Most of the API in part two is a Draw.io-specific way of reaching these few concepts.

3.1. Nodes, connections and what they stand for

Diagrams show **things** and their relationships with **shapes** joined by **lines**. I call the shapes **nodes**, the lines **connections**, and the things they stand for **entities**. Entities have information associated with them, which I call **features**, and they can do things, which I call **operations**.

Think of entities as nouns, features as adjectives and operations as verbs. In "the quick brown fox jumps over the lazy dog", the fox and the dog are entities; quick, brown and lazy are features; jumps over is an operation, perhaps with "over" as its argument.

A **diagram** contains **diagram elements**, nodes and connections, and describes a **domain**, a collection of entities with their features and operations. A few facts about this structure matter throughout the book:

- A node can contain other diagram elements. A system contains its containers, a container its components.
- A connection joins a source node to a target node, but either end can be missing. A dangling connection is legitimate: an arrow from a system to "somewhere we have not decided yet".
- A diagram element can stand for one entity, for a group of entities, for a feature or an operation, or for nothing at all. A frame, a legend or a decorative line is a purely visual element.
- The relationship between an element and what it stands for is not in the picture. It is established by the shape, the label, the tooltip, a link, a property, and the shared understanding of the people who read it.

Features come in two kinds. **Attributes** have scalar values, or collections of them: a name, a color, an age. **References** point to other entities: a person's employer, the systems a team owns. On a diagram, a reference is often a connection, and sometimes containment: the teams inside a department, the containers inside a system.

3.2. Visual features

Diagram elements have features of their own, apart from what they stand for, and the visual ones can carry information as surely as a label. Most diagram editors support visual features only, and Draw.io adds non-visual ones, properties, which Chapter 8 is about. Each visual feature below can encode both categories and quantities.

Label. Text on an element or a connection, plain or formatted. In Draw.io, labels are often HTML, which matters when a program reads them (Chapter 7).

Shape. Circle, rectangle, cylinder, cloud. Circles for apples and rectangles for oranges is a category; the ratio of a rectangle's sides can be a quantity.

Icon. An image used instead of a shape. There are millions of icons, and Draw.io has libraries of thousands, from cloud providers to network equipment.

Border. Width, color and pattern: solid, dashed, dotted. In this book's figures, a dashed border means planned.

Connection. Width, color and pattern, as for a border, and also decorations at both ends and labels along the line.

Size. Lines of code, account balance, transaction volume: the bigger the shape, the larger the number.

Geometry. Position and distance. Position can order nodes, left to right and top to bottom, or clockwise around a hub, and distance can carry a value. Chapter 12 turns geometry into order with comparators.

Color. Colors can be ordered, by hue on the HSV or HSB scale, or put in buckets, by distance to a set of base colors: blue for in progress and green for done, as in many issue trackers. Matching colors exactly is brittle; matching by least distance is not.

Opacity. A translucent element can be less important, or not available yet.

Gradient. A gradient between two colors can say that an element belongs to two categories.

Z-order. When shapes overlap, which one is on top can carry meaning, alone or together with

position.

A diagram that uses visual features to carry information should say so, in a legend or in its documentation. A diagram that a program reads can do better: it can read those features, because in Draw.io every one of them is in the element’s style, which the API reads as a map (Chapter 11).

Chapter 4. Why Draw.io

There are many diagram editors. This book builds on Draw.io, also known as diagrams.net, for reasons that have little to do with drawing and much to do with what happens to a diagram after it is drawn.

4.1. What it gives you

Draw.io is a diagram editor written in HTML5 and JavaScript, developed by JGraph, open source under the Apache License 2.0, and free.

- **It runs everywhere.** In the browser, as a desktop application for Windows, macOS and Linux, inside Confluence and Visual Studio Code, and self-hosted as a web application or a container.
- **It stores diagrams where you already keep files.** The local disk, Google Drive, OneDrive, Dropbox, GitHub and GitLab. A diagram is a file, versioned and reviewed like any other.
- **It has a large library of shapes**, more than two thousand, and accepts images pasted from anywhere.
- **It has a viewer.** A diagram embedded in a web page can be clicked, zoomed, shown full screen, its layers and tags toggled, and opened in the editor, without a Draw.io installation.
- **Elements have tooltips, editable IDs and properties**, in the *Edit Data* dialog, so an element can carry information that is not drawn.
- **A file has pages, and elements link to pages**, so one file holds a hierarchy of diagrams, from a system context down to the code.
- **Layers and tags** show and hide elements in groups, so one diagram serves several audiences or reveals its details step by step in a presentation.
- **Placeholders** put property values into labels and tooltips.
- **Libraries and templates** make reusable sets of elements, preconfigured with names, tooltips and links, the beginning of a diagramming ecosystem in an organization.

4.2. What it does not give you

What Draw.io lacks is what the rest of the continuum needs. A tooltip is too small for documentation. Links between pages stop at the file: you cannot link to an element, or to a page in another file. And there is no programmatic way, outside the browser, to read a diagram, to navigate it as a structure, to check it, or to change it.

That is what the module provides, and, in the runtime's plans, what the points to its right build on (Chapter 14).

4.3. The file format

A Draw.io file is XML, and its structure is the structure the API exposes (Chapter 6):

- `mxfile`, the document, contains one `diagram` element per page;
- a `diagram` contains an `mxGraphModel`, with the page's settings;
- the model contains a `root`, which contains `mxCell` elements: a root cell, the layers, and the nodes and connections of each layer, each with a `parent` attribute;
- a cell with properties is wrapped in an `object` element, which holds the properties, the label, the tooltip and the link as attributes;
- a cell's `style` attribute holds its visual features as `key=value;` pairs, and an `mxGeometry` child holds its position, size and the points of a connection.

The content of a page may be **compressed**: deflated, Base64-encoded text inside the `diagram` element, which is how the desktop application saves by default. Compressed files are smaller; uncompressed ones can be read, diffed and merged. The API reads both and writes either (Chapter 5).

4.4. Diagrams in PNG images

Draw.io can save a diagram **inside** a PNG image: "Include a copy of my diagram" on export, or a file named `.drawio.png`, which the editor opens and saves directly. The result is one file that is an ordinary image to everything that shows images, and a diagram to Draw.io.

Every figure in this book is such a file. A figure cannot drift away from its source, because the figure is the source. The module reads the diagram out of the image, so the book's tests load the

very figures you see, and a listing that works on a figure works on what the figure shows.

Two notes for your own diagrams:

- **Export above 100 percent.** A PNG is raster. A diagram exported at 1x looks soft in print and on high-density screens; this book's figures are rendered at 2x.
- **Prefer PNG to SVG for documents built with Asciidoctor PDF.** Draw.io can embed a diagram in SVG as well, but it writes formatted labels as HTML inside `foreignObject` elements, which Asciidoctor PDF does not draw.

4.5. The module

The module is a salvage of the Draw.io API that Nasdanika core has had for years. It keeps the API for reading, navigating, changing and writing documents, and leaves out what tied it to the rest of core: graph processing, the Ecore model of Draw.io documents, which the runtime has rebuilt as a module of its own, and Spring Expression Language (Appendix B).

It is small, and it depends on little: the JDK's XML and image APIs, four small libraries for JSON, HTML, text and encoding, and the runtime's `common` module, which brings EMF's core libraries and the OpenTelemetry API. You can use it without the rest of the runtime.

Part II: The How

The Java API, one task at a time, on one running example: loading and saving diagrams, the document model, navigation, properties, selectors, links, changing diagrams and putting their elements in order. Then rendering pages as images and videos with Draw.io's own viewer in a browser.

Chapter 5. Getting started

This chapter adds the module to a project, loads the book's running example, and saves, shares and reloads it in the forms the API supports.

5.1. Adding the module

The runtime publishes a bill of materials, `org.nasdanika.sdk.runtime:bom`, which manages the versions of its modules and of the libraries they use. Import it, then depend on `org.nasdanika.sdk.runtime:drawio` without a version:

Listing 1. Importing the runtime's bill of materials

```
<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.nasdanika.sdk.runtime</groupId>
            <artifactId>bom</artifactId>
            <version>${project.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```



The listing is the BOM import of the SDK book's examples, which use `${project.version}` because they are built in the runtime's reactor. A project of your own uses the runtime's version, 2026.11.0, in its place. Until the runtime is on Maven Central, build it from github.com/Nasdanika-SDK/runtime with `mvn clean install` first.

The module is a named Java module. Its declaration shows what it exports and what it brings with it:

Listing 2. The module declaration of `drawio`

```
module org.nasdanika.sdk.runtime.drawio {

    exports org.nasdanika.sdk.runtime.drawio;
    exports org.nasdanika.sdk.runtime.drawio.comparators;
    exports org.nasdanika.sdk.runtime.drawio.impl;
    exports org.nasdanika.sdk.runtime.drawio.style;
```

```
    exports org.nasdanika.sdk.runtime.drawio.style.impl;
    exports org.nasdanika.sdk.runtime.drawio.util;
    requires transitive org.nasdanika.sdk.runtime.common;
    requires transitive java.xml;
    requires java.desktop;
    requires org.json;
    requires org.jsoup;
    requires org.apache.commons.codec;
    requires org.apache.commons.text;

}
```

Your own `module-info.java` needs `requires org.nasdanika.sdk.runtime.drawio;`. The module requires `common` transitively, which brings the runtime's foundation with it: EMF's `URI` class, used for document locations throughout the API, and the OpenTelemetry API. `java.xml` is transitive too, because `Element.getElement()` returns the DOM element behind any part of a diagram. `java.desktop` is there only for reading PNG images.

5.2. The running example

Most listings in this book work on one diagram, drawn after the Internet Banking System example of the C4 model:

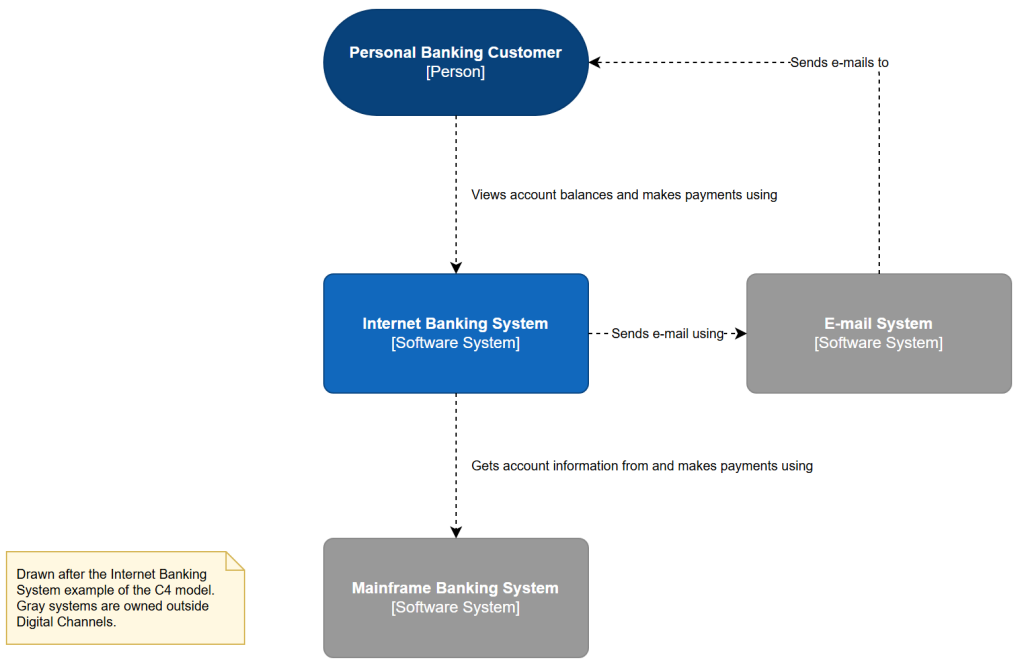


Figure 2. The running example: the System Context page of `internet-banking.png`

It has two pages. *System Context*, in the figure, has a person, three software systems and the connections between them, and a *Notes* layer with the note in the corner. Its elements carry properties, such as `type`, `owner` and `protocol`, and tags, such as `internal` and `legacy`. *Containers*, the second page, shows what is inside the Internet Banking System: a single-page application, an API application and a database, inside a boundary that links back to the system on the first page. A PNG image shows only the first page of the diagram it embeds; open the file in Draw.io to see the second.

5.3. Loading a diagram

A PNG image can embed more than one diagram, so `Document.loadFromPngMetadata` returns a list. The tests load the figures with a small helper:

Listing 3. Loading a diagram embedded in a PNG image (`Diagrams.java`)

```
/**
 * Loads the diagram embedded in a PNG image next to this class.
 */
public static Document load(String name) throws Exception {
    URL resource = Diagrams.class.getResource(name);
    List<Document> documents = Document.loadFromPngMetadata(resource); ①
    if (documents.size() != 1) {
        throw new IllegalArgumentException(name + " embeds " + documents.size() + " diagrams");
    }
    return documents.getFirst();
}
```

① `loadFromPngMetadata` takes a `URL`, or an `InputStream` with the URI of the image, and optionally a `Context` (Chapter 8).

The first listing loads the running example and lists its pages:

Listing 4. A first diagram (`GettingStartedTests.java`)

```
Document document = Diagrams.load("internet-banking.png");
List<String> pageNames = document
    .getPages()
    .stream()
    .map(Page::getName)
    .toList();
assertThat(pageNames).containsExactly("System Context", "Containers");
```

5.4. Saving and loading

`save` writes the document as XML. Its argument decides compression: `false` writes pages as plain XML, `true` compresses them, and `null` keeps each document as it was loaded.

Listing 5. Saving a diagram as a `.drawio` file and loading it back

```
Path path = Path.of("target", "book", "internet-banking.drawio");
Files.createDirectories(path.getParent());
Files.writeString(path, document.save(false)); ①

Document reloaded = Document.load(path.toFile()); ②
assertThat(reloaded.getPages()).hasSize(2);
assertThat(reloaded.getURI().lastSegment()).isEqualTo("internet-banking.drawio");
```

① Uncompressed, indented XML, which reads well in a diff and in a code review.

② The document remembers where it was loaded from. Relative links to other documents are resolved against that URI (Chapter 10).

A compressed document is smaller and opaque. Loading from a string needs a base URI only if the document links to other documents:

Listing 6. Compressed pages, and loading from a string

```
String compressed = document.save(true);
assertThat(compressed).doesNotContain("mxGraphModel"); ①
Document fromString = Document.load(compressed, null); ②
assertThat(fromString.getPageById("containers").getName()).isEqualTo("Containers");
```

① The pages are deflated and Base64-encoded, so the XML of the model is not visible.

② The URI is `null`: the document has no location of its own.

The API loads documents from every source Java has. `Document.create` makes an empty one, which Chapter 11 fills:

Listing 7. Every way to get a document

```
File file = new File(
    "src/test/resources/org/nasdanika/sdk/runtime/drawio/tests/compressed.drawio");
Document fromFile = Document.load(file);
Document fromUrl = Document.load(file.toURI().toURL());
Document fromUri = Document.load(URI.createFileURI(file.getAbsolutePath()));
Document fromStream;
try (var in = Files.newInputStream(file.toPath())) {
    fromStream = Document.load(in, URI.createFileURI(file.getAbsolutePath()));
```

```
}
Document empty = Document.create(false, null);
```

Each `load` method also takes a `Context`, which supplies properties, filters them, and opens the streams of linked documents (Chapter 8, Chapter 10). Loading from a `URL` or a `File` goes through a cache of documents, so documents that link to each other are loaded once each.

5.5. Sharing a diagram

Most people who should see a diagram do not have Draw.io. `toHtml` produces a `div` with the diagram and a `script` element for Draw.io's viewer, ready to be put into any web page:

Listing 8. A web page with the diagram in Draw.io's viewer

```
String html = document.toHtml(); ①
Path page = Path.of("target", "book", "internet-banking.html");
Files.createDirectories(page.getParent());
Files.writeString(page, "<html><body>" + html + "</body></html>");
assertThat(html).contains("class=\"mxgraph\"").contains(Document.JSDELIVR_DRAWIO_VIEWER);
```

① Without arguments, the diagram is compressed into the page and the viewer is loaded from jsDelivr, pinned to a release of Draw.io. `toHtml(compress, viewer)` takes another viewer URL, or `null` for no `script` element when the page loads the viewer itself.

The viewer shows a toolbar for zoom, layers, tags and full screen, highlights links, and opens the diagram in the editor in a new window.

A diagram can also travel as a URI. `toDataURI` encodes the document as a `data:` URI, which `Document.load` decodes, so a diagram fits wherever a URI does: in a property, a configuration file, a message or a link:

Listing 9. A diagram as a `data:` URI

```
URI dataUri = document.toDataURI(true); ①
assertThat(Document.isDataURI(dataUri)).isTrue();
Document fromDataUri = Document.load(dataUri); ②
assertThat(fromDataUri.getPages()).hasSize(2);
```

① `true` compresses the pages inside the URI.

② `Document.isDataURI` tells such a URI apart from a location.

Chapter 6. The document model

The API is a set of interfaces over the XML of a Draw.io file, one per part of it, implemented in the `impl` package. This chapter is the map; the chapters after it use it.

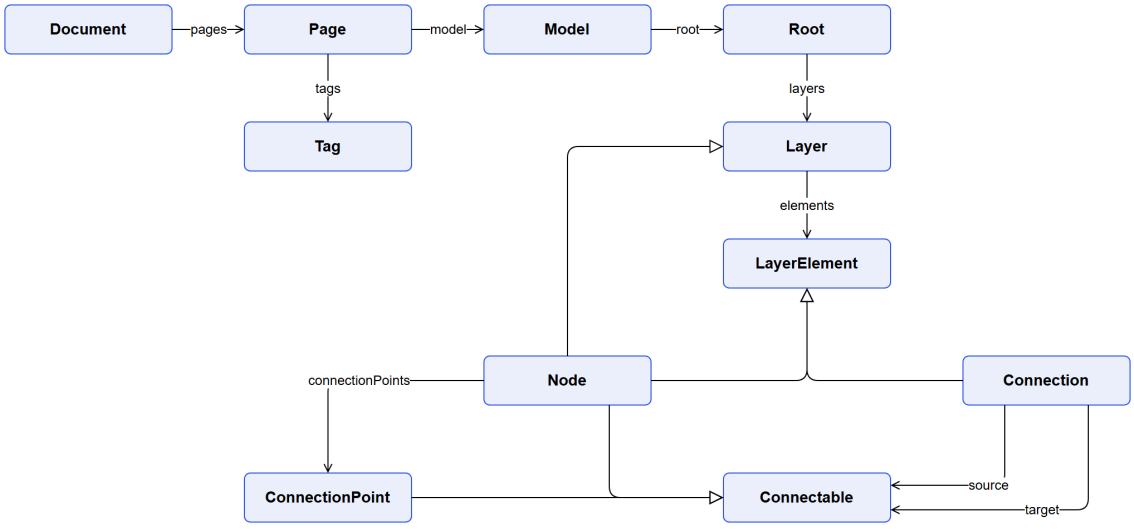


Figure 3. The document model. Open arrows are references, named after their getters; closed arrows are generalizations.

6.1. From document to element

`Document` is the file, the `mxfile` element. It has a list of pages, creates new ones, finds a page by name or by ID, saves itself, and loads documents that it links to.

`Page` is a `diagram` element: a name, an ID and a model. A page is also where tags live.

`Model` is the page's `mxGraphModel`: the grid, the background, the shadow and the page view. It has one `Root`, the root cell, which contains the layers. For most purposes a page, its model and its root are one thing; the API keeps them apart because the format does.

`Layer` contains the elements of a page. Every page has at least one layer; in Draw.io, the *Layers* panel shows them.

`LayerElement` is what a layer contains: a node or a connection.

`Node` is a shape. A node is also a layer: it contains nodes and connections of its own, which is how a system contains its containers. It has a geometry and connection points, and it is `Connectable`: connections start and end at nodes.

`Connection` is a line from a source to a target, either of which can be missing. It has points along its path and a node of its own for each extra label.

`ConnectionPoint` is a fixed point on the outline of a node where a connection leaves or arrives, and is connectable too.

`Tag` is a name that elements of a page share, so that they can be shown and hidden together.

The figure leaves out three interfaces that the others extend. `Element` is the base of everything above: it has a URI, a marker that says where it came from in its file, the DOM element behind it, and the visitor and stream methods of Chapter 7. `ModelElement` is the base of the root, layers, nodes and connections: an ID, a label, a tooltip, a link, a style, properties, tags and visibility, and the selectors of Chapter 9. `LinkTarget` marks what a link can point to: a page or a model element (Chapter 10).

6.2. The figure checks the API

The figure is a diagram that a test reads. Its node labels are interface names, its open arrows are labeled with the names of getters, and its closed arrows have a `relationship` property of `extends`. The test checks the figure against the API with reflection, so a renamed interface or getter fails the build of this book until the figure is redrawn:

Listing 10. Checking the figure against the API (`MetamodelTests.java`)

```
@Test
public void testFigureMatchesApi() throws Exception {
    Document metamodel = Diagrams.load("metamodel.png");
    List<Node> types = metamodel
        .stream()
        .filter(Node.class::isInstance)
        .map(Node.class::cast)
        .toList();
    assertThat(types).hasSize(11);
    for (Node type: types) {
        assertThat(apiType(type).isInterface()).as(type.getLabel()).isTrue(); ①
    }

    List<Connection> relationships = metamodel
```

```
            .stream()
            .filter(Connection.class::isInstance)
            .map(Connection.class::cast)
            .toList();
        for (Connection relationship: relationships) {
            Class<?> source = apiType((Node) relationship.getSource());
            Class<?> target = apiType((Node) relationship.getTarget());
            switch (relationship.getProperty("relationship")) {
            case "extends" -> assertThat(target.isAssignableFrom(source)) ②
                .as(source.getSimpleName() + " extends " + target.getSimpleName())
                .isTrue();
            case "reference" -> {
                String name = relationship.getLabel();
                String getter = "get" + Character.toUpperCase(name.charAt(0)) + name.substring(1);
                Method method = source.getMethod(getter); ③
                assertThat(method.getGenericReturnType().getTypeName())
                    .as(source.getSimpleName() + "." + getter + "()")
                    .contains(target.getSimpleName());
            }
            default -> throw new AssertionError("Unknown relationship " + relationship);
            }
        }
    }

    private static Class<?> apiType(Node node) throws ClassNotFoundException {
        return Class.forName(Document.class.getPackageName() + "." + node.getLabel());
    }
```

① Every node names an interface of the API's package.

② A generalization arrow is an `extends`: the source type is assignable to the target type.

③ A reference arrow is a getter of its source whose return type mentions its target. `getPages()` returns `List<Page>`, `getTags()` on a page returns `Map<String, Tag>`.

It is a small example of the continuum at work. The diagram is documentation for people and a specification for a test, and the test keeps the documentation honest.

6.3. Types and generics

The interfaces are generic in themselves: `Element<E extends Element<E>>`, `ModelElement<M extends ModelElement<M>>`, `Layer<L extends Layer<L>>`. The parameter lets methods such as `style` return the element's own type, and in client code it is almost always a wildcard: `ModelElement<?>`, `Layer<?>`. `Node` and `Connection` are not generic, so code that works with them reads as plain Java.

A model element that is a node or a connection is a `LayerElement`, and you tell them apart with `instanceof`. Streams of a document are streams of `Element<?>`, filtered and cast to what a listing

needs, as the next chapter shows.

`getElement()` returns the `org.w3c.dom.Element` behind any part of a diagram. It is there for what the API does not cover, such as attributes of the model that have no getter. Changes made through the DOM are saved like any others.

Chapter 7. Navigating a diagram

A program that reads a diagram walks it, searches it, follows its connections and folds it into a result. The API has a way to do each.

7.1. Walking the containment

Pages contain a model, the model a root, the root layers, and layers their elements. A loop over that structure is the most direct way to read a diagram:

Listing 11. Walking pages, layers and elements (`NavigationTests.java`)

```
List<String> outline = new ArrayList<>();
for (Page page: document.getPages()) {
    outline.add(page.getName());
    for (Layer<?> layer: page.getModel().getRoot().getLayers()) {
        outline.add("  layer " + layer.getId());
        for (LayerElement<?> element: layer.getElements()) {
            outline.add("    " + element.getId());
        }
    }
}
assertThat(outline).startsWith(
    "System Context",
    "  layer 1",
    "    customer",
    "    ibs",
    "    mainframe",
    "    email",
    "    uses-ibs");
assertThat(outline).contains("  layer notes", "    legend", "Containers", "    ibs-boundary");
```

The layer with ID `1` is the default layer that Draw.io creates on every page; the *Notes* layer of the running example has the ID `notes`. Elements come in the order of the file, which is the z-order of the page: what was drawn later is on top. `getElements()` returns the elements directly in a layer; a node's own `getElements()` returns what it contains, because a node is a layer.

7.2. Streams

`stream()` returns an element and everything it contains, depth first, as a `Stream<Element<?>>`. On a document, that is every page, model, layer, node and connection in it. Filtering by type and property is the everyday way to find things:

Listing 12. Finding the software systems by a property

```
List<String> systems = document
    .stream()
    .filter(Node.class::isInstance)
    .map(Node.class::cast)
    .filter(node -> "software-system".equals(node.getProperty("type")))
    .map(Diagrams::text)
    .toList();
assertThat(systems).containsExactly(
    "Internet Banking System [Software System]",
    "Mainframe Banking System [Software System]",
    "E-mail System [Software System]");
```

Labels with `html=1` in their style are HTML. The running example's labels have a bold name and a line break before the C4 type, so the listing reads them through a helper that keeps the text:

Listing 13. The text of a label (`Diagrams.java`)

```
/**
 * Labels of elements with html=1 in their style are HTML. This is the text a reader sees.
 */
public static String text(ModelElement<?> element) {
    String label = element.getLabel();
    return label == null ? null : Jsoup.parse(label).text();
}
```

The helper uses jsoup, which the module requires as well; add it to your own project if you need it.

Two shortcuts search a whole document or page for a single element:

Listing 14. Finding an element by ID and by property

```
ModelElement<?> ibs = document.getModelElementById("ibs");
ModelElement<?> core = document.getModelElementByProperty("owner", "Core Banking");
assertThat(core.getId()).isEqualTo("mainframe");
assertThat(ibs.getModel().getPage().getName()).isEqualTo("System Context");
```

`getModelElementsByProperty(name, value)` returns all of them. IDs are unique within a page, not within a document; `getModelElementById` on a document returns the first match in page order. Draw.io generates IDs such as `7KSC1_O8d7ACaxm1iSCq-1`, and lets you edit them: right-click, *Edit Data*, and click the ID. Readable IDs, as in the running example, make diagrams far easier to work with from code, and worth the minute they take.

7.3. Connections

A node knows the connections that start and end at it. `getOutgoingConnections()` and `getIncomingConnections()` return the ones attached to the node itself; `getAllOutgoingConnections()` and `getAllIncomingConnections()` add the ones attached to its connection points (Chapter 11).

Listing 15. Following connections

```
Node ibs = (Node) document.getModelElementById("ibs");
Map<String, String> uses = ibs
    .getOutgoingConnections()
    .stream()
    .collect(Collectors.toMap(
        Connection::getLabel,
        connection -> Util.getConnectableNode(connection.getTarget()).getId())); ①
assertThat(uses).containsOnly(
    Map.entry("Gets account information from and makes payments using", "mainframe"),
    Map.entry("Sends e-mail using", "email"));

assertThat(ibs.getIncomingConnections())
    .extracting(Connection::getId)
    .containsExactly("uses-ibs");
```

① A connection's source and target are `Connectable`: a node or a connection point. `Util.getConnectableNode` returns the node in both cases.

7.4. Visitors

`accept` visits an element and its children bottom up: children first, then the element itself, with a map from each child to the result of visiting it. That makes a visitor a fold, in which every element computes its result from its children's:

Listing 16. Counting nodes with a visitor

```
BiFunction<Element<?>, Map<? extends Element<?>, Integer>, Integer> counter =
    (element, childResults) -> {
        int nodes = element instanceof Node ? 1 : 0;
        for (Integer childResult: childResults.values()) { ①
            nodes += childResult;
        }
        return nodes;
    };
assertThat(systemContext.accept(counter)).isEqualTo(5); ②
assertThat(document.getPageByName("Containers").accept(counter)).isEqualTo(5);
```

① The results of the children. `null` results are left out of the map.

② Four nodes on the default layer and the note on the *Notes* layer.

`accept` also takes a `Consumer<Element<?>>` and a connection base, for visitors that collect rather than compute.

7.5. Connection base

Draw.io puts a connection in a layer, or in the node that contains both of its ends. For many purposes that parent is not the logical one. In a diagram of systems, a connection from the Internet Banking System to the mainframe is something the Internet Banking System does, so it belongs under its **source**. In a mind map, where arrows point from a topic to its parent, a connection belongs under its **target**.

`ConnectionBase` says which:

- `PARENT`, the default, keeps connections where the file puts them;
- `SOURCE` makes each connection a child of its source node;
- `TARGET` makes each connection a child of its target node.

The connection base applies to visitors, to `stream(ConnectionBase)` and to URI resolution (Chapter 10):

Listing 17. The same page under two connection bases

```
BiFunction<Element<?>, Map<Element<?>, String>, String> tree =
    (element, childResults) -> {
        String children = String.join(", ", childResults.values());
        String id = element instanceof ModelElement<?> me ? me.getId() : "page";
        return children.isEmpty() ? id : id + "(" + children + ")";
    };
String byParent = systemContext.accept(tree, ConnectionBase.PARENT);
String bySource = systemContext.accept(tree, ConnectionBase.SOURCE); ①
assertThat(bySource).contains("ibs(ibs-mainframe, ibs-email)");
assertThat(bySource).contains("customer(uses-ibs)");
assertThat(byParent).doesNotContain("ibs(ibs-mainframe");
```

① Under `SOURCE`, connections leave the layer and appear under the nodes they start at.

7.6. Tags and layers

Tags are names in an element's `tags` property, separated by spaces, and Draw.io's *Edit Tags* dialog and the viewer's tag filter work with them. A page collects the tags of its elements:

Listing 18. Tags of a page and of an element

```
assertThat(systemContext.getTags().keySet())
    .containsExactlyInAnyOrder("person", "internal", "legacy", "note"); ①

Tag legacy = systemContext.getTags().get("legacy");
assertThat(legacy.getElements())
    .extracting(ModelElement::getId)
    .containsExactly("mainframe");

ModelElement<?> mainframe = document.getModelElementById("mainframe");
assertThat(mainframe.getTags())
    .extracting(Tag::getName)
    .containsExactlyInAnyOrder("internal", "legacy");
```

① `getTags()` on a page is computed from its elements, so it has exactly the tags in use. `createTag(name)` makes a tag to add to an element's `getTags()` set, which writes the `tags` property.

Layers have a label, which Draw.io's *Layers* panel shows, and visibility. Hiding a layer hides it in the editor and in the viewer, which makes layers a way to keep notes, annotations or work in progress in a diagram without showing them:

Listing 19. Hiding a layer

```
Layer<?> notes = systemContext
    .getModel()
    .getRoot()
    .getLayers()
    .stream()
    .filter(layer -> "Notes".equals(layer.getLabel()))
    .findFirst()
    .orElseThrow();
assertThat(notes.isVisible()).isTrue();
notes.setVisible(false); ①
assertThat(document.save(false)).contains("visible=\"0\"");
```

① `setVisible` works on any model element, not only on layers. An invisible element is still in the document and still found by streams and visitors.

Chapter 8. Properties and placeholders

Properties are what turn a picture into data. In Draw.io, select an element, choose *Edit Data* from its context menu (Ctrl+M or Cmd+M), and add a property by name. The file stores it as an attribute of the element's `object` element, next to the label, the tooltip and the link.

8.1. Reading properties

Listing 20. Properties of the Internet Banking System (`PropertiesTests.java`)

```
ModelElement<?> ibs = document.getModelElementById("ibs");
assertThat(ibs.getProperty("type")).isEqualTo("software-system");
assertThat(ibs.getProperty("owner")).isEqualTo("Digital Channels");
assertThat(ibs.getPropertyNames()) ①
    .containsExactlyInAnyOrder("id", "type", "owner", "placeholders");
assertThat(ibs.getLink()).isEqualTo("data:page/id,containers");
assertThat(ibs.getProperty("budget")).isEmpty(); ②
```

① `getPropertyNames()` returns the names of the attributes that hold properties, `id` and `placeholders` included. The attributes with getters of their own, `label`, `tooltip`, `link`, `tags`, `style` and `visible`, are left out.

② A property that is not set is an empty string, not `null`. Check with `isEmpty()` or `Util.isBlank`.

The label, the tooltip and the link are properties too: `getProperty("tooltip")` returns the same as `getTooltip()`. A property name that contains a slash is a property path, which reads a property of a related element (Chapter 9).

8.2. Placeholders

Draw.io replaces `%name%` in labels and tooltips with the value of the element's property `name`, when the element has *Placeholders* checked in its *Edit Data* dialog, which sets the property `placeholders` to `1`. The API does the same, in labels and in every property it returns:

Listing 21. Placeholders, and the style of an element as properties

```
assertThat(ibs.getTooltip()).isEqualTo( ①
    "Allows customers to view information about their bank accounts and make payments."
    + " Owned by Digital Channels.");

ibs.setProperty("colors", "Filled with %$style:fillColor%"); ②
```

```
assertThat(ibs.getProperty("colors")).isEqualTo("Filled with #1168BD");
```

① The tooltip in the file is "… Owned by %owner%.", and `owner` is a property of the same element.

② `%$style:key%` is the value of a key of the element's style. Here the property `colors` is set from code, then read back with its placeholder expanded.

A placeholder is resolved in this order:

1. `$style:key`, from the element's style;
2. if the document was loaded with an `EvaluatingContext`, the context evaluates the name, which is a hook for an expression language of your choice;
3. otherwise the element's own property of that name;
4. if that is empty, the same property of the element's parent, its parent's parent, and so on up to the root;
5. and finally the document's context, which can supply properties of its own.

Values are expanded recursively, and a placeholder that refers to itself, directly or through others, is expanded once. A placeholder that resolves to nothing is left as it is, so a missing value shows in the result rather than disappearing. A placeholder names a property, not a property path: `%../owner%` does not read the parent's owner (Chapter 9).

8.3. Properties from outside the diagram

The last step of the resolution is the `Context` the document was loaded with. A context's `getProperty` supplies values that are not in the diagram at all: the environment, the release, the region. `Context.fromPropertySource` makes one from any `Function<String, String>`, and a map's `get` is one:

Listing 22. Properties from the context

```
Map<String, String> environment = Map.of("environment", "production");
Context context = Context.fromPropertySource(environment::get); ①
Document document;
try (InputStream in = Diagrams.class.getResourceAsStream("internet-banking.png")) {
    List<Document> documents = Document.loadFromPngMetadata(in, null, context);
    document = documents.getFirst();
}

ModelElement<?> api = document.getModelElementById("api");
api.setProperty("placeholders", "1");
```

```
api.setProperty("endpoint", "https://api.%environment%.example.com"); ②
assertThat(api.getProperty("endpoint")).isEqualTo("https://api.production.example.com");
```

① Every `load` method has a variant that takes a context.

② The API Application inherits nothing from its parents, so `%environment%` comes from the context.

The same diagram loaded with another context reads differently, and nothing in the file changes. That is a way to use one diagram for several environments, or for several releases, with links and labels that follow them.

8.4. Filtering properties

A context also sees every property as it is read, and can change it. `filterProperty` receives the element, the name and the value, and returns the value to use. Filters suit cross-cutting rules, such as redaction for a diagram published outside the team:

Listing 23. A context that hides owners

```
Context publicRendering = new Context() {

    @Override
    public String filterProperty(ModelElement<?> element, String name, String value) {
        return "owner".equals(name) ? null : value; ①
    }

};
Document document;
try (InputStream in = Diagrams.class.getResourceAsStream("internet-banking.png")) {
    document = Document.loadFromPngMetadata(in, null, publicRendering).getFirst();
}
ModelElement<?> ibs = document.getModelElementById("ibs");
assertThat(ibs.getProperty("owner")).isNull();
assertThat(ibs.getProperty("type")).isEqualTo("software-system");
```

① The filter sees every value read through `getProperty`, the tooltip and the link included, before its placeholders are expanded. Labels, and the values that placeholders pull in, are read without it.

Contexts compose: `Context.from(a, b)` and `a.compose(b)` ask `a` first and fall back to `b`, run both filters in turn, and open streams with whichever context can.

8.5. Writing properties

`setProperty` writes a property, and setting it to `null` or an empty string removes it. An element that has no properties yet is a bare `mxCell` in the file; the first property wraps it in an `object` element, as Draw.io does.

Listing 24. Writing properties

```
ModelElement<?> mainframe = document.getModelElementById("mainframe");
mainframe.setProperty("retirement", "2028");
mainframe.setTooltip(mainframe.getTooltip() + ". Retirement planned for 2028");
mainframe.setProperty("owner", null); ①
assertThat(mainframe.getPropertyNames()).contains("retirement").doesNotContain("owner");
assertThat(document.save(false)).contains("retirement=\"2028\"");
```

① `null` removes the property.

Writing properties from code is how a program puts what it learned back into the diagram, where people see it: an owner looked up in a directory, a retirement date from a portfolio, a status from a deployment. Chapter 11 does the same with styles.

Chapter 9. Selectors and property paths

Streams and loops find elements anywhere. Selectors find them **relative to an element**: its parent, its children, the elements its connections lead to, and what it links to. A selector is a path of segments separated by slashes, and `ModelElement.select` returns the elements at its end.

9.1. Segments

Segment	Selects	On
`..`	The parent.	Any model element
`child`	The children: for a node, the nodes and connections it contains.	Any model element
`link-target`	What the element's link points to: an element, or the root of a linked page (Chapter 10).	Any model element
`incoming-connection`	The connections that end at the node or at its connection points.	Nodes
`outgoing-connection`	The connections that start at the node or at its connection points.	Nodes
`source`	The source node of the connection.	Connections
`target`	The target node of the connection.	Connections

`child`, `incoming-connection` and `outgoing-connection` take a qualifier in square brackets. `[id]` keeps the elements with that ID; `[name=value]` keeps the elements whose property `name` equals `value`. Matching is exact and case-sensitive, with no wildcards.

9.2. Following connections

Listing 25. Selecting along connections (`SelectorTests.java`)

```
ModelElement<?> customer = document.getModelElementById("customer");
assertThat(customer.select("outgoing-connection/target")) ①
    .extracting(ModelElement::getId)
    .containsExactly("ibs");

ModelElement<?> ibs = document.getModelElementById("ibs");
```

```
assertThat(ibs.select("outgoing-connection[protocol=SMTP]/target")) ②
    .extracting(ModelElement::getId)
    .containsExactly("email");
assertThat(ibs.select("incoming-connection/source"))
    .extracting(ModelElement::getId)
    .containsExactly("customer");
```

① From the customer, along its outgoing connections, to their targets.

② Only the connection whose `protocol` is `SMTP`.

9.3. Containment

Listing 26. Selecting parents and children

```
ModelElement<?> api = document.getModelElementById("api");
ModelElement<?> boundary = api.select("..").getFirst();
assertThat(boundary.getId()).isEqualTo("ibs-boundary");
assertThat(boundary.select("child"))
    .extracting(ModelElement::getId)
    .contains("spa", "api", "database");
assertThat(boundary.select("child[technology=Java]"))
    .extracting(ModelElement::getId)
    .containsExactly("api");
```

9.4. Across pages

`link-target` follows the element's link, so a selector crosses from one page to another, and from one document to another, wherever the diagram's links lead:

Listing 27. From the Containers page back to the System Context page

```
assertThat(boundary.select("link-target/outgoing-connection/target")) ①
    .extracting(ModelElement::getId)
    .containsExactlyInAnyOrder("mainframe", "email");
```

① The boundary on the Containers page links to the Internet Banking System on the System Context page; from there, the selector follows its outgoing connections.

9.5. Property paths

A property name with slashes is a property path: a selector, then the name of a property. `getProperty` selects with all but the last segment, and reads the last one from the first element

selected:

Listing 28. Property paths

```
ModelElement<?> ibs = document.getModelElementById("ibs");
assertThat(ibs.getProperty("outgoing-connection[protocol=SMTP]/target/owner"))
    .isEqualTo("Workplace Services");
assertThat(ibs.getProperty("incoming-connection/protocol")).isEqualTo("HTTPS");

ModelElement<?> spa = document.getModelElementById("spa");
assertThat(spa.getProperty("../link-target/owner")).isEqualTo("Digital Channels"); ①
assertThat(spa.getProperty("outgoing-connection/target/technology")).isEqualTo("Java");
```

① From the single-page application to its parent, the boundary, to what the boundary links to, the Internet Banking System, and its `owner`.

Property paths work with `getProperty`, and so with everything built on it, such as the property comparator of Chapter 12. They do not work inside placeholders: `%../owner%` in a label is not expanded (Chapter 8).

9.6. When nothing matches

A selector that matches nothing returns an empty list, and a property path that selects nothing returns `null`. An unknown segment is not an error: it matches nothing. The exception is `link-target` on an element without a link, which returns `null` rather than an empty list; check `getLink()` or `isTargetLink()` first.

Listing 29. Selectors that match nothing

```
assertThat(ibs.select("child")).isEmpty();
assertThat(ibs.select("outgoing-connection[protocol=FTP]")).isEmpty();
assertThat(ibs.getProperty("outgoing-connection[protocol=FTP]/target/owner")).isNull();
```



Three defects found while writing this chapter, each reproduced against 2026.11.0:

- `select("link-target")` on an element without a link returns `null`, not an empty list, and a property path through it, such as `getProperty("link-target/owner")`, throws a `NullPointerException`. `ModelElementImpl.selectSegment` returns `null` for an unresolved `link-target`; it should return an empty list.

- `..` on the root returns a list that contains `null`, so a property path that climbs past the root throws a `NullPointerException`.
- Placeholders read raw attributes and fall back to the parent's `getProperty`, so a placeholder that is a property path, such as `%../id%`, is either left unexpanded or resolved against the parent and its ancestors, and can throw for the reason above. Either support paths in placeholders or reject them with a clear message.

Separately, a property that is not set reads as an empty string, while a property path that selects nothing reads as `null` (Chapter 8). Code that reads both has to check for both. Consider returning `null` for both, or document the difference in the Javadoc of `ModelElement.getProperty`.

Chapter 10. Links between pages, elements and documents

Links are how a diagram gets levels of detail: an element on a system context page links to the page that shows what is inside it. Draw.io links an element to a URL or to a page of the same file. The API reads those links, and extends them to elements, and to pages and elements of other documents, so a set of files becomes one network of diagrams.

10.1. Link syntax

Draw.io writes a link to a page as `data:page/id,<page ID>`. The API accepts four forms, two of which are Draw.io's own:

Link	Points to
`data:page/id,<page>`	A page by ID. Draw.io writes this form when you choose a page in the *Edit Link* dialog.
`data:page/name,<page>`	A page by name, URL-encoded: `data:page/name,System+Context`.
`data:element/id,<element>`	An element by ID.
`data:element/name,<element>`	An element by the text of its label, HTML removed, URL-encoded.

The part after the comma can say where to look:

- `<document>#<page>` for a page in another document, where `<document>` is a URI resolved against the URI of the linking document: `data:page/name,overview.drawio#Page+2`;
- `[<document>#]<page selector>/<element>` for an element on another page, where the page selector is `id,<page ID>` or `name,<page name>`: `data:element/id,id,system-context/ibs`, `data:element/name,overview.drawio#name,Page+2/Linked`.

An element link without a page selector looks on the linking element's own page. Element links and links to other documents are an extension: Draw.io itself opens them as URLs, so they work in programs built on the API and not when clicked in the editor or the viewer.

10.2. Resolving links

`getLinkTarget()` returns what a link points to: a `Page` or a `ModelElement`, both `LinkTarget`s. `isTargetLink()` says whether the link is in one of the four forms at all; a link to a web page returns `false`, and its target is `null`.

Listing 30. Links within the running example (`LinkTests.java`)

```
ModelElement<?> ibs = document.getModelElementById("ibs");
assertThat(ibs.isTargetLink()).isTrue();
Page containers = (Page) ibs.getLinkTarget(); ①
assertThat(containers.getName()).isEqualTo("Containers");

ModelElement<?> boundary = document.getModelElementById("ibs-boundary");
assertThat(boundary.getLinkTarget()).isSameAs(ibs); ②

ModelElement<?> mainframeRef = document.getModelElementById("mainframe-ref");
Node mainframe = (Node) mainframeRef.getLinkTarget(); ③
assertThat(mainframe.getModel().getPage().getName()).isEqualTo("System Context");
assertThat(mainframe.getId()).isEqualTo("mainframe");
```

① The Internet Banking System links to the *Containers* page, by ID, as Draw.io wrote it.

② The boundary on the *Containers* page links back to the Internet Banking System, by page ID and element ID.

③ The Mainframe Banking System on the *Containers* page links to its namesake on the *System Context* page, by page name and element ID.

A link that cannot be resolved, to a page or an element that does not exist, throws a `ConfigurationException` with a marker that names the page and the element that has the link. That makes checking a diagram's links a one-line test: visit every element with `isTargetLink()` and call `getLinkTarget()`.

10.3. Following links

Visitors and streams stay within the element they start from. `Util.withLinkTargets` wraps a visitor so that it also visits what elements link to, each link target once:

Listing 31. Visiting linked pages

```
List<String> pages = new ArrayList<>();
Consumer<Element<?>> visitor = element -> {
    if (element instanceof Page page) {
```

```
        pages.add(page.getName());
    }
};
systemContext.accept(visitor, null);
assertThat(pages).containsExactly("System Context");

pages.clear();
systemContext.accept(Util.withLinkTargets(visitor, null), null); ①
assertThat(pages).containsExactlyInAnyOrder("System Context", "Containers");
```

① The wrapped visitor visits the *Containers* page because the Internet Banking System links to it.

`Util.traverser` goes further: it visits connected elements and parents too, and with them everything reachable from the starting element.

10.4. Other documents

A link to another document loads it, with the same context, and caches it, so documents that link to each other are loaded once each and links between them come back to the same objects:

Listing 32. A link to an element in another document

```
URL links = Diagrams.class.getResource("/org/nasdanika/sdk/runtime/drawio/tests/links.drawio");
Document document = Document.load(links);
Node linkSource = document
    .stream()
    .filter(Node.class::isInstance)
    .map(Node.class::cast)
    .filter(node -> "Link to linked on compressed second page".equals(node.getLabel()))
    .findFirst()
    .orElseThrow();
assertThat(linkSource.getLink())
    .isEqualTo("data:element/name,compressed.drawio#name,Page+2/Linked");

Node linked = (Node) linkSource.getLinkTarget(); ①
Document other = linked.getModel().getPage().getDocument();
assertThat(other.getURI().lastSegment()).isEqualTo("compressed.drawio");
assertThat(other).isSameAs(document.getDocument(URI.createURI("compressed.drawio"))); ②
```

① `compressed.drawio` is resolved against the URI of `links.drawio`, loaded, and searched for an element labeled *Linked* on its page *Page 2*.

② `getDocument(uri)` returns a linked document from the same cache, so this is the document the link was resolved in.

With cross-document links, several teams can each keep their own diagrams in their own

repositories, and the enterprise's diagrams become one network: a system context diagram owned by enterprise architects links to container diagrams owned by the teams that build the systems.



A linked document is always parsed as XML, so a link cannot point to a diagram embedded in a PNG image, such as the figures of this book. `DocumentImpl.getDocument` could recognize PNG images by their signature, or by the `.png` extension, and load them with `loadFromPngMetadata`. Until it does, diagrams that link to each other have to be `.drawio` files.

10.5. Logical URIs

The URIs in links do not have to be locations. A context's `openStream(URI)` opens every linked document, so it can map logical URIs to wherever the documents are: a classpath, a Maven artifact, a GitLab repository read through its API without cloning, or a different location for each environment:

Listing 33. Logical URIs resolved by the context

```
Context catalog = new Context() {

    @Override
    public InputStream openStream(URI uri) throws java.io.IOException,
            java.net.URISyntaxException {
        if ("catalog".equals(uri.scheme())) { ①
            String path = "/org/nasdanika/sdk/runtime/drawio/tests/" + uri.opaquePart();
            return Diagrams.class.getResourceAsStream(path);
        }
        return Context.super.openStream(uri);
    }

};

Document document = Document.create(false, null, catalog);
Layer<?> layer = document.createPage().getModel().getRoot().getLayers().getFirst();
Node node = layer.createNode();
node.setLink("data:page/name,catalog:compressed.drawio#Page+2"); ②
LinkTarget<?> target = node.getLinkTarget();
assertThat(((Page) target).getName()).isEqualTo("Page 2");
```

① URIs with the scheme `catalog` are read from the classpath; everything else is opened as a URL.

② The link uses the logical URI. Nothing in the diagram says where the document is.

With placeholders on, a link can also depend on the context: `data:page/name,%release%/overview.drawio#Overview`, with `release` supplied by the context, links each release's diagram to that release's overview.

10.6. Element URIs

`getURI()` returns a URI for every element: the document's URI with a fragment of the IDs from the page down, so it identifies the element across documents. `resolveURI(base, provider, connectionBase)` builds URIs of your own, from a property of each element resolved against its logical parent's, with the connection base of Chapter 7 deciding whether a connection's parent is its layer, its source or its target. That is how a diagram maps to a hierarchy of resources, such as the pages of a documentation site, or the paths of an API.

Chapter 11. Creating and changing diagrams

A program that writes diagrams puts what it knows where people look. It can generate a diagram from scratch, from a system, an inventory or a model, or change one that people drew, to mark what a check found or what a deployment did, and leave the rest of the drawing as it was.

11.1. Creating a diagram

A new document has no pages; a new page has a model, a root and one layer. Nodes are created in a layer, or in another node:

Listing 34. A new diagram with two nodes (`CreatingTests.java`)

```
Document document = Document.create(false, null); ①
Page page = document.createPage();
page.setName("Payments");
Layer<?> layer = page.getModel().getRoot().getLayers().getFirst(); ②

Node api = layer.createNode();
api.setId("api");
api.setLabel("API Application");
api.setProperty("technology", "Java");
api.getGeometry().setBounds(0, 0, 160, 60);
api.getStyle()
    .backgroundColor("#438DD5")
    .fontColor("#FFFFFF")
    .rounded(true);

Node gateway = layer.createNode();
gateway.setId("gateway");
gateway.setLabel("Payment Gateway");
gateway.getGeometry().setBounds(300, 0, 160, 60);
gateway.getStyle().backgroundColor("#999999").fontColor("#FFFFFF").rounded(true);
gateway.getTags().add(page.createTag("external"));
```

① An uncompressed document with no URI. `Document.create(compressed, uri, context)` takes a context as well.

② The default layer. `root.createLayer()` adds another.

A node created by the API gets a generated ID; `setId` replaces it with a readable one. Its

geometry is relative to its parent: to the page for a node in a layer, to the containing node otherwise.

11.2. Styles

An element's style is the `style` attribute in the file, a list of `key=value;` pairs, and `getStyle()` returns it as a live `Map<String, String>`. Changing the map changes the attribute. `Style`, `LineStyle`, `NodeStyle` and `ConnectionStyle` add typed, fluent methods for common keys on top of the map:

- `Style`: `opacity`, `rounded`, `shadow`, `enumerate` and `enumerateValue`;
- `LineStyle`: `color`, the stroke, `width` and `dashed`;
- `NodeStyle`: `backgroundColor`, the fill, `shape`, `align`, `verticalAlign`, `fontSize`, `fontColor`, `fontStyle`, `labelBackgroundColor`, `labelBorderColor`, `rotation`, `container` and `collapsible`;
- `ConnectionStyle`: `edgeStyle`, `startArrow`, `endArrow`, `startFill`, `endFill` and `jump`, with the `Arrow` and `Jump` enums for the common values.

The typed methods are a convenience, not a complete list. Draw.io has hundreds of style keys, and any of them can be put into the map directly: `style.put("gradientColor", "#7EA6E0")`. The easiest way to find the key for an effect is to apply it in Draw.io and look at *Edit Style*.

11.3. Connections

Listing 35. A connection between the two nodes

```
Connection payment = layer.createConnection(api, gateway);
payment.setId("api-gateway");
payment.setLabel("Submits payments to");
payment.setProperty("protocol", "ISO 20022/HTTPS");
payment.getStyle()
    .edgeStyle("orthogonalEdgeStyle")
    .dashed("1")
    .endArrow(Arrow.CLASSIC)
    .endFill(true);
```

`createConnection(source, target)` takes nodes or connection points, and either can be `null` for a dangling end. Connections can have points of their own, and they can leave and enter nodes at fixed connection points, which Draw.io stores in the connection's style as `exitX`, `exitY`, `entryX` and `entryY`:

Listing 36. Connection points and waypoints

```
ConnectionPoint exit = api.createConnectionPoint(1, 0.5); ①
ConnectionPoint entry = gateway.createConnectionPoint(0, 0.5);
Connection status = layer.createConnection(gateway, api);
status.setLabel("Reports status to");
status.setSource(gateway.createConnectionPoint(0.5, 1));
status.setTarget(api.createConnectionPoint(0.5, 1));
status.getPoints().add(380, 100); ②
status.getPoints().add(80, 100);
payment.setSource(exit);
payment.setTarget(entry);
```

① A connection point is a position relative to the node's size: `(1, 0.5)` is the middle of the right side. An overload takes offsets in pixels too, and whether the point is on the shape's perimeter.

② Waypoints, in page coordinates, which Draw.io routes the line through.

`Connection.createNode()` adds a label to a connection, as a node of its own with a relative geometry along the line.

11.4. Saving and reading back

Listing 37. Saving the new diagram and reading it back

```
Path path = Path.of("target", "book", "payments.drawio");
Files.createDirectories(path.getParent());
Files.writeString(path, document.save(false));

Document reloaded = Document.load(path.toFile());
Node reloadedApi = (Node) reloaded.getModelElementById("api");
assertThat(reloadedApi.getAllOutgoingConnections()) ①
    .extracting(ModelElement::getLabel)
    .containsExactly("Submits payments to");
assertThat(reloadedApi.getProperty("outgoing-connection/target/tags"))
    .isEqualTo("external");
```

① `getAllOutgoingConnections()` includes connections that leave from the node's connection points; `getOutgoingConnections()` would not.

The file opens in Draw.io as drawn. What it does not have is a picture: a document written by a program has no rendering until something renders it. A diagram generated by code and shown in documentation, as a PNG with the diagram embedded, needs a renderer in the build; Chapter 14 says how the runtime will provide one.

11.5. Changing a diagram that people drew

The most useful writes are often small. A program that knows something about the elements of a diagram can show it on the diagram, and leave the layout, the wording and everything else that people decided alone. Nasdanika core called this representation filtering: the source diagram stays as it is, and the diagram people see shows the current state of affairs. Systems under maintenance turn translucent, systems with an incident get a red border, planned systems are gray, and readers grasp the state of a landscape at a glance.

Listing 38. Marking legacy systems

```
document
    .stream()
    .filter(Node.class::isInstance)
    .map(Node.class::cast)
    .filter(node -> node.getTags().stream().anyMatch(tag -> "legacy".equals(tag.getName())))
    .forEach(node -> node.getStyle() ①
        .color("#B91C1C")
        .width("3")
        .dashed("1"));

String saved = document.save(false);
assertThat(saved).contains("strokeColor=#B91C1C");
```

① Three keys of the node's style: `strokeColor`, `strokeWidth` and `dashed`.

11.6. Removing elements

`remove()` removes an element and everything it contains from the document. It removes only that: the connections that start or end at a node stay where they are, with an end that no longer resolves.

Listing 39. Removing a node leaves its connections

```
document.getModelElementById("email").remove();
Connection sendsEmail = (Connection) document.getModelElementById("ibs-email");
assertThat(sendsEmail).isNotNull(); ①
assertThat(sendsEmail.getTarget()).isNull();
```

① The connection is still in the document, and its target is `null`. Draw.io shows it as a dangling line.

To remove a node the way Draw.io's *Delete* does, remove its connections first:

Listing 40. Removing a node and its connections

```
Node email = (Node) document.getModelElementById("email");
for (Connection connection: email.getAllIncomingConnections()) {
    connection.remove(); ①
}
for (Connection connection: email.getAllOutgoingConnections()) {
    connection.remove();
}
email.remove();
assertThat(document.getModelElementById("email")).isNull();
assertThat(document.getModelElementById("ibs-email")).isNull();
Node ibs = (Node) document.getModelElementById("ibs");
assertThat(ibs.getOutgoingConnections())
    .extracting(Connection::getId)
    .containsExactly("ibs-mainframe");
```

① Both lists are snapshots, so removing connections while iterating over them is safe.



`NodeImpl.onRemove()` calls `clear()` on `getAllIncomingConnections()` and `getAllOutgoingConnections()`, which looks intended to remove the node's connections with it. The lists are copies, so `clear()` has no effect, and `remove()` leaves dangling connections, as the listing above shows. Decide which behavior is intended: removing the connections, as Draw.io's *Delete* does, or keeping them, as `Delete All` with *Edges* unchecked would. Then make `onRemove` do it, or delete it, and update this section. `layer.getElements().remove(element)` is a second way to remove an element; check that it behaves the same.

Chapter 12. Geometry and order

The elements of a diagram come in the order of the file, which is the order in which they were drawn. That order rarely means anything. The order people see does: the steps of a process clockwise around a hub, the stages of a pipeline from left to right, the numbered steps of a sequence, the flow of the arrows. A program that turns a diagram into a list, a table of contents, a sequence of slides, or the steps of a plan needs that order, and the module's comparators compute it.

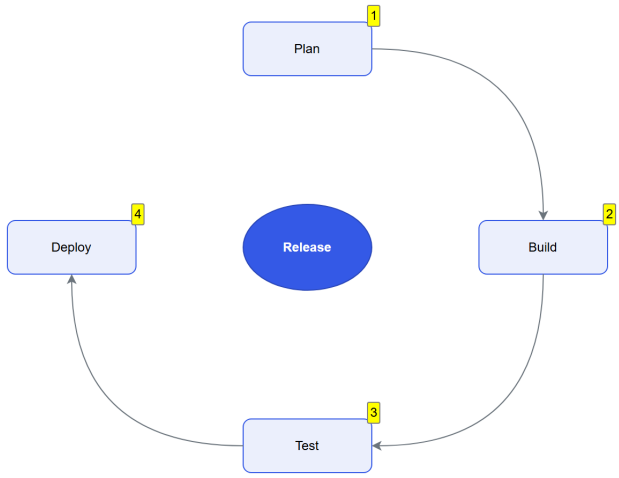


Figure 4. A release cycle, drawn as people draw cycles

The figure has a hub, four steps around it, arrows from step to step, and the numbered badges that Draw.io draws for elements with *Enumerate* checked in their style. Each step also has an `owner` property.

12.1. Orders that are not geometry

The listings sort the four steps with a small helper:

Listing 41. Sorting nodes and returning their labels (`OrderTests.java`)

```
private static List<String> sort(List<Node> nodes, Comparator<? super Node> comparator) {
    return nodes.stream().sorted(comparator).map(Node::getLabel).toList();
}
```

Listing 42. Orders by label, property, flow and enumeration

```
Node release = (Node) document.getModelElementById("release");
List<Node> steps = document
    .stream()
    .filter(Node.class::isInstance)
    .map(Node.class::cast)
    .filter(node -> node != release)
    .toList();

assertThat(sort(steps, new LabelModelElementComparator()))
    .containsExactly("Build", "Deploy", "Plan", "Test");
assertThat(sort(steps, new PropertyModelElementComparator("owner")))
    .containsExactly("Test", "Build", "Deploy", "Plan"); ①
assertThat(sort(steps, new FlowComparator()))
    .containsExactly("Plan", "Build", "Test", "Deploy"); ②
assertThat(sort(steps, new EnumerateComparator()))
    .containsExactly("Plan", "Build", "Test", "Deploy");
```

① By the `owner` property: Assurance, Engineering, Operations, Product.

② By the arrows: an element comes before the elements it leads to.

`LabelModelElementComparator` compares the text of labels, HTML removed, and `PropertyModelElementComparator` compares a property, both with empty values last. `FlowComparator` counts the steps along connections from one element to the other, so it orders a chain by its arrows; a predicate restricts the connections it follows. `EnumerateComparator` compares Draw.io's enumeration values as paths of numbers and words, so `1.10` comes after `1.9` and before `2`.

12.2. Geometry

Listing 43. Orders by angle and by position

```
assertThat(sort(steps, new AngularNodeComparator(release, null))) ①
    .containsExactly("Plan", "Deploy", "Test", "Build");
assertThat(sort(steps, new AngularNodeComparator(release, null).reversed()))
    .containsExactly("Build", "Test", "Deploy", "Plan");
assertThat(sort(steps, new CartesianNodeComparator(Direction.rightDown))) ②
    .containsExactly("Deploy", "Plan", "Test", "Build");
assertThat(sort(steps, new CartesianNodeComparator(Direction.downRight)))
    .containsExactly("Plan", "Deploy", "Build", "Test");
```

① By angle around the hub, starting at twelve o'clock and going counterclockwise, as angles do in mathematics.

② By columns from left to right, then by rows from top to bottom. Nodes that overlap horizontally are in the same column.

`AngularNodeComparator` takes the hub and a base angle in radians, which defaults to π/2, twelve o'clock. Reversing it gives a clockwise order, but one that starts just before the base angle, so the step at twelve o'clock comes last; to start a clockwise order at twelve o'clock, reverse it with a base angle a little past twelve o'clock. `CartesianNodeComparator` has eight directions: `rightDown` and `downRight` above, and the other combinations of right or left with down or up. Geometry is absolute: a node inside a container is compared by its position on the page.

12.3. Enumeration

Draw.io's *Enumerate* checkbox puts a badge on an element, and the badge shows a number that Draw.io computes, or the value of `enumerateValue` when the style has one.

Listing 44. Enumeration in the style

```
ModelElement<?> plan = document.getModelElementById("plan");
assertThat(plan.getStyle().enumerate()).isTrue();
assertThat(plan.getEnumarateValue()).isEqualTo("1");
```

`getEnumarateValue()` returns the value of the badge, or `null` without one. The method name has a typo, kept for compatibility with Nasdanika core.

12.4. Comparators that are not orders

`FlowComparator` and `EnumerateComparator` do not define a total order on every set of elements. Two elements that no path connects are equal to the flow comparator, and so are two elements without enumeration values to the enumeration comparator, which treats them as equal to everything. Java's sorting assumes a consistent order, and on such sets it gives an order that depends on the input. Chain them with a comparator that is total, `thenComparing(new PositionModelElementComparator())` for the order of the file or the label comparator, and keep cycles out of what you sort by flow.

12.5. The continuum, again

The figure of Chapter 2 is a diagram whose arrows are its order. Sorting its points by flow gives the chapter's sections in order, and their `status` property says what this module does:

Listing 45. The points of the continuum, in order (`OrderTests.java`)

```
Document continuum = Diagrams.load("continuum.png");
List<Node> points = continuum
    .stream()
    .filter(Node.class::isInstance)
    .map(Node.class::cast)
    .filter(node -> node.getProperty("role").isEmpty()) ①
    .toList();
assertThat(sort(points, new FlowComparator())).containsExactly(
    "Picture",
    "Interactive diagram",
    "Documented diagram",
    "Diagram as data",
    "Executable diagram",
    "Diagram as a model");
assertThat(points)
    .filteredOn(point -> "available".equals(point.getProperty("status")))
    .extracting(Node::getLabel)
    .containsExactlyInAnyOrder("Interactive diagram", "Diagram as data");
```

① The legend has a `role` property; the points do not.

`Comparators` lists keys for all of these orders, such as `clockwise`, `flow`, `right-down` and `reverse-enumerate`, for diagrams and configurations that choose an order by name. In this module the keys are only names; code that reads them chooses the comparator.



The Javadoc of `AngularNodeComparator`'s constructor documents a `clockwise` parameter that does not exist and calls the base angle degrees, while the code uses radians. The comparator orders counterclockwise, as the listing shows. Fix the Javadoc, or add the parameter, and consider a factory that maps the keys of `Comparators` to comparators, which Nasdanika core had in its mapping module.

Chapter 13. Rendering in a browser

A diagram that a program built or changed has no picture until something draws it (Chapter 11). In Draw.io that does not matter, because the editor draws a diagram as it opens it. A documentation build, a report or a video needs the picture as a file. This chapter renders pages as PNG and SVG images, clicks shapes before taking the picture, and records videos of pages shown in sequence.

13.1. Why draw.io's own viewer

A `.drawio` file does not hold everything that Draw.io shows. Draw.io computes much of it each time it draws:

- **Edge routes**, from the style. An orthogonal edge often stores few points or none, and line jumps and loops are computed too.
- **Where an edge meets a shape**, from the shape's outline and the edge's exit and entry constraints.
- **Labels**, which are usually HTML that the browser wraps.

Drawing a diagram in Java would mean rebuilding all of that, and keeping up with Draw.io as it changes. Instead, the module runs Draw.io's own viewer, the script that shows diagrams embedded in web pages, in a browser that Playwright starts and drives. The picture is what the person who drew the diagram saw, because it is drawn by the same code. The price is a browser.

13.2. The module

Rendering is in a module of its own, `org.nasdanika.sdk.runtime:drawio-playwright`, so that the `drawio` module stays light: Playwright and a browser come only with this one. It is in this book because it is about Draw.io documents.

Listing 46. The module declaration of `drawio-playwright`

```
module org.nasdanika.sdk.runtime.drawio.playwright {

    requires transitive org.nasdanika.sdk.runtime.drawio;
    requires transitive org.nasdanika.sdk.runtime.playwright;
    requires org.json;
```

```
    exports org.nasdanika.sdk.runtime.drawio.playwright;

}
```

Your `module-info.java` needs `requires org.nasdanika.sdk.runtime.drawio.playwright;`, which brings the Draw.io API and the runtime's Playwright module with it.

A rendering launches Chromium, in one of two ways:

- With a **channel**, a browser installed on the machine, such as `chrome` or `msedge`. Nothing is downloaded.
- Without one, the **Chromium that Playwright downloads** on first use into a cache in the user's home directory, a few hundred megabytes, once.

The tests take the channel from the `nasdanika.playwright.channel` system property, which the build sets from the `playwright.channel` Maven property, `chrome` by default. They are skipped if no browser can be launched. Videos also need FFmpeg, which Playwright installs on request:

```
mvn exec:java -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="install ffmpeg"
```

The listings work on a document with three pages: *Context* and *Containers*, a smaller variant of this book's running example, and *Payment*, with a button that runs custom actions.

13.3. A page as PNG

Listing 47. Rendering the first page (`TestDrawioPlaywright.java`)

```
Document document = Document.load(TestDrawioPlaywright.class.getResource("banking.drawio"));
Path file = rendering().png(document, OUTPUT.resolve("context.png")); ①

try (InputStream in = Files.newInputStream(file)) {
    List<Document> embedded = Document.loadFromPngMetadata(in, null); ②
    assertThat(embedded).singleElement()
        .extracting(d -> d.getPages().getFirst().getName())
        .isEqualTo("Context");
}
```

① The first page of the document, cropped to the diagram and a border of 10 pixels, written to the file.

② The image carries the page, as Draw.io's own PNG export does with *Include a copy of my*

diagram: `loadFromPngMetadata` reads it back, and Draw.io opens the image for editing.

The embedded document has only the page in the picture. A person who opens the image in Draw.io edits that page, and saves the image again with the page in it, which is what the figures of this book are.

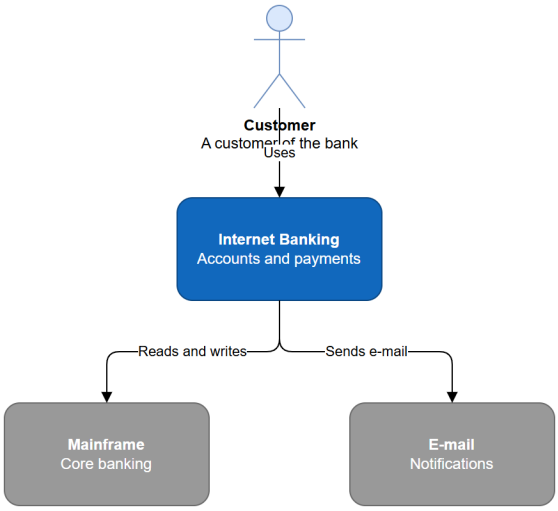


Figure 5. The Context page at scale 2 (`rendering-context.png`)

The Draw.io API also writes the embedding directly: `document.embedInPng(png, compress)` adds the document to any PNG image, after its header, without changing the image data.

13.4. A page as SVG

Listing 48. Rendering a page as SVG

```
Page containers = document.getPageByName("Containers");
Path file = rendering().svg(containers, OUTPUT.resolve("containers.svg"));

try (InputStream in = Files.newInputStream(file)) {
    Document embedded = Document.loadFromSvg(in, null); ①
    assertThat(embedded.getPages()).singleElement().extracting(Page::getName).isEqualTo
("Containers");
}
```

① The SVG carries the page in the `content` attribute of its root element, as Draw.io's editable SVG export does. `Document.loadFromSvg` reads it.

The SVG is the one Draw.io exports: shapes and edges as SVG elements with `data-cell-id`

attributes naming their cells, and HTML labels as `foreignObject` elements. Browsers draw `foreignObject`. Some other renderers do not, among them the one Asciidoctor PDF uses, so a PDF takes the PNG.

13.5. Several pages

Listing 49. Rendering pages by name, at scale 2

```
List<Page> pages = Pages.byName(document, "Context", "Containers"); ①
List<Path> files = rendering()
    .withScale(2) ②
    .png(pages, OUTPUT.resolve("pages"), Rendering.fileNames(".png")); ③
assertThat(files).extracting(f -> f.getFileName().toString()).containsExactly("context.png",
"containers.png");
```

① Pages by name, in the order given. `Pages.byId` picks them by ID, and `Pages.all` takes them all. A missing page is an error, not a gap.

② Twice as many pixels in each direction, through the browser's device scale factor. The diagram is drawn at that resolution, not enlarged afterwards.

③ A file per page, named by a function of the page: `Rendering.fileNames(".png")` names a page's file after its name, in lower case with words joined by hyphens.

The pages are rendered in one browser session: the viewer is loaded once and shows each page in turn. `svg` takes the same arguments.

13.6. Interacting before the picture

A page can be changed in the browser before its picture is taken. The last argument of `png` and `svg` is a function that receives the viewer and the page once the viewer shows the page:

Listing 50. Clicking a button that runs custom actions

```
Path directory = OUTPUT.resolve("payment");
Rendering rendering = rendering().withScale(2);
List<Page> pages = Pages.byName(document, "Payment");
List<Path> files = rendering.png(pages, directory, Rendering.fileNames("-after.png"), (viewer,
page) -> {
    viewer.png(directory.resolve("payment-before.png")); ①
    viewer.click("pay-button"); ②
    viewer.session().pause(Duration.ofMillis(1500)); ③
});
```

① A picture of the page as it is first shown. The function can take as many as it likes.

② A click in the middle of the shape of the cell with the ID `pay-button`, as a person would click it. The button's link holds custom actions, which start flow animations on the two connections and color the ledger.

③ Time for the actions to run. The pause is in the browser session's API, with the rest of Playwright.

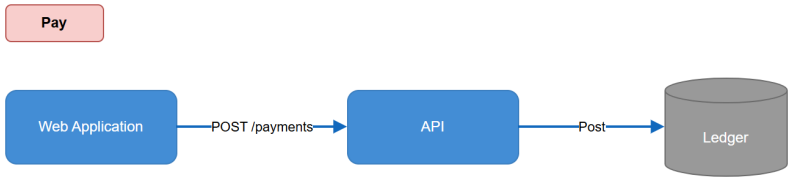


Figure 6. Before the click (`rendering-payment-before.png`)

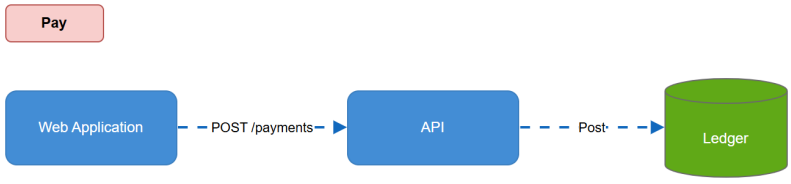


Figure 7. After the click (`rendering-payment-after.png`)

The function receives a `Viewer`. `viewer.session()` is the runtime's Playwright session, with Playwright's page, steps, screenshots and narration, and the viewer itself is in the browser's global variable `nasdanikaViewer` for scripts that need more of Draw.io's API.

Method	Returns or does
`page()`, `document()`	The page shown, and its document
`show(page)`	Shows another page of the document
`click(id)`, `click(element)`	Clicks the shape of a cell, in its middle
`png()`, `png(file)`	The page shown as a PNG image, with the page embedded
`svg()`, `svg(file)`	The page shown as SVG, with the page embedded
`session()`	The browser session

`rendering.inViewer(document, viewer → ⋯)` opens a document in the viewer and hands the viewer over, for anything the methods above do not cover.

13.7. Videos

A video shows pages in sequence, each for as long as its frame says:

Listing 51. A tour of three pages

```
Page context = document.getPageByName("Context");
Page containers = document.getPageByName("Containers");
Page payment = document.getPageByName("Payment");
Path video = rendering().video(OUTPUT.resolve("tour.webm"), List.of(
        Frame.of(context, Duration.ofSeconds(2)), ①
        Frame.of(containers, Duration.ofSeconds(2)),
        Frame.of(payment, Duration.ofSeconds(3), (viewer, page) -> viewer.click("pay-button"))));
②
assertThat(video).isNotEmptyFile();
```

① A frame: a page and how long it is shown.

② A frame with a function, which runs once the page is shown: here it clicks the button, and the actions play while the frame lasts.

The video is WebM, at the size of the browser's viewport, 1280 by 800 pixels by default. Each page is fit to the viewport and centered, enlarged at most twice. The recording starts when the browser opens its page, so the video begins with the fraction of a second the viewer takes to load.

Draw.io animations are what videos are for: flow animations along connections, and custom actions that show, hide, fade, highlight and restyle shapes. In the viewer they run only when a person clicks, and in a video they run on cue, the same way every time.

13.8. Rendering options

`Rendering` is a record with a `with` method per component, and `Rendering.DEFAULT`:

Component	Default	Meaning
`openTelemetry`	the current one	Receives the spans and log records of the sessions

Component	Default	Meaning
`browser`	Playwright's Chromium, headless, 1280 by 800, scale 1, no step screenshots	The browser session's options; `withScale` sets the scale of PNG images
`viewer`	Draw.io's viewer 32.5.5 from jsDelivr	The URL of the viewer script
`border`	10	Space around the diagram in PNG and SVG images, in pixels
`embed`	`true`	Embed the page in PNG and SVG images

Each rendering is a browser session in a `Browser` span of the trace, and each page a step span named after the page. With a video, each step span carries its offset into the video, so a trace and a video can be read together. The runtime's Playwright module and its Zephyr Book describe sessions, steps and the trace in full.

13.9. Limits

- **The viewer comes from jsDelivr**, at a pinned version, so rendering needs the network. It is downloaded once per JVM and served to the browser from memory. Shipping the viewer as a resource, for builds without the network, is PLANNED.
- **Fonts are the machine's.** Draw.io's default font is Helvetica, and a browser draws it with whatever it substitutes: Arial on Windows, Liberation Sans on Linux. The same page can come out a few pixels different on two machines, so tests compare embedded documents, not pixels.
- **What the viewer does not draw is not in the picture**: shapes from stencil libraries that the viewer does not include, or images that cannot be loaded.



The runtime's figure tool, `doc/zephyr/tools/drawio-figure`, renders the first page of a document with a shell script, Chrome's screenshot option and a Java program that crops the image and embeds the document. This module does the same from Java, so the tool can be retired once the books' figures are rendered with it.

Part III: Looking Ahead

The points of the continuum that this module does not reach yet, and how the runtime will reach them.

Chapter 14. The rest of the continuum

This module covers the middle of the continuum: a diagram as data, read, checked, changed and written by programs, and shared as an interactive page. The points to its right exist in Nasdanika core, built on its graph, mapping and HTML application modules. In the runtime they will be rebuilt on its own foundation, and this chapter says how, as far as it is designed. Everything in it is **PLANNED** unless it says otherwise.

14.1. Documented diagrams

A documentation site generated from a diagram: a page per documented element, a navigation tree from the diagram's containment and links, the diagrams embedded in the viewer and linked to the pages, and search. In Nasdanika core, the `drawio html-app site` command of the Nasdanika CLI, and a GitHub template repository that runs it, generate such sites, from documentation in properties, from Markdown files that elements refer to, and from prototypes that mount other sites.

In the runtime, sites will be generated with its HTML and Bootstrap modules, which its own site is planned to be generated with as well. The diagram supplies the tree, the element properties supply the pages, and the API of this book reads both.

14.2. Executable diagrams

Behavior bound to diagram elements: a property names the code an element stands for, and the code is wired to its neighbors along the diagram's connections. Nasdanika core does this with graph processors, which exchange handlers and endpoints along connections, synchronously or concurrently, and are created from invocable URIs in element properties.

The design for the runtime is open. Its natural foundation is the runtime's capability framework, which finds and creates the code that a requirement names, with the diagram as the wiring, and its telemetry, which would record what the diagram did.

14.3. Diagrams as models

Diagram elements mapped to the objects of a typed model, so that a diagram is one of the sources of a model alongside spreadsheets, scripts and code. In Nasdanika core, semantic mapping does this: element properties name the classes and features of an Ecore metamodel, and a diagram loaded into a resource set is a model.

The runtime takes a different road there, and the first part of it is built, in modules beside this one **AVAILABLE**:

- **The Draw.io model**, `models/drawio`, is an EMF model of Draw.io documents. A `.drawio` file, or a PNG or SVG image that Draw.io exported with the diagram in it, loads into any resource set of the runtime as a model, through a resource factory that the module contributes, and saves back. It reads and writes with the API of this book. Its Zephyr Book is *Draw.io Model*.
- **The diagram model**, `models/diagram`, which the Draw.io model extends, gives every element semantic elements: references to the objects of other models that the element depicts. Its Zephyr Book, *Diagram Model*, maps the figure of its own metamodel, drawn in Draw.io, to that metamodel and checks one against the other.

What remains **PLANNED** is the mapping itself without code: element properties that name classes and features, as in core, so that a diagram loaded into a resource set is a model of the domain and not only of the diagram. Until then, a mapping is a few lines of Java over the diagram model, or a Groovy script: the runtime's Groovy DSL can read a Draw.io resource while it builds a model.

14.4. One diagram model, many pictures

Most of this is built **AVAILABLE**. The runtime's diagram model holds what notations have in common: nodes, ports, connections, layers, nesting and labels, independent of any editor. A Draw.io document is one: its pages are slides of the presentation model, and slides are diagrams.

- **Layout**, with Eclipse ELK, is written once for the diagram model, in `models/diagram/elk`, and lays out Draw.io pages because they are diagrams. The *Draw.io Model* book generates a diagram of the modules of a running JVM, lays it out with ELK and renders it with Draw.io's viewer (Chapter 13).
- **Rendering** goes to React Flow, where a diagram should be interactive, to Draw.io's own

viewer, where a diagram should look exactly as drawn, and to SVG, through the runtime's SVG model, where it should be vector graphics in a document. The Zephyr Books *Diagram Layout with ELK* and *SVG Model* show them.

PLANNED: routes in from other notations. The presentation model, for slides, is a sketch, and models of Visio, PowerPoint and Excel files are postponed.

The same path makes diagrams scenes of narrated videos: a diagram on screen, with elements highlighted as a narration mentions them, regenerated when the diagram changes **PLANNED**.

14.5. Rendering generated diagrams

Rendering is built: Chapter 13 renders pages with Draw.io's own viewer, as PNG and SVG images with the page embedded and as videos. What remains **PLANNED** is an Asciidoctor extension that renders a `.drawio` file referenced from a book as the book is built, so that the source of a figure in Git is the diagram rather than its image, and the viewer shipped as a resource, so that rendering does not need the network.

14.6. Diagrams and AI assistants

A diagram as data is good context for an assistant: a graph of short texts, labels, tooltips and properties, joined by containment, connections and links. An assistant that is given the elements near the one a person asks about, by graph distance as well as by similarity of text, answers questions about an architecture more precisely than one given the whole diagram as an image. The API of this book is enough to build that context today; the runtime's plans for assistants will build it in.

Appendix A: API reference

A summary of the API by package, for looking things up. The Javadoc of the module is the complete reference.

A.1. `org.nasdanika.sdk.runtime.drawio`

Type	What it is for
`Document`	The file. Static `load` methods for strings, readers, streams, files, URLs and URIs, `data:` URIs included; `loadFromPngMetadata` for PNG images; `loadFromSvg` for SVG images with the document in the `content` attribute; `create` for a new document. Instance methods: `getPages`, `createPage`, `getPageByName`, `getPageById`, `save`, `toHtml`, `toDataURI`, `embedInPng`, `getDocument(URI)` for linked documents, `getProperty` from the context.
`Document.Context`	What a document is loaded with: `openStream(URI)` for linked documents, `getProperty(name)` for placeholders, `filterProperty(element, name, value)`. Factories `fromPropertySource`, `fromURIHandler`, `fromURIConverter`; `from(⋯)`, `compose` and `reduce` to combine contexts.
`Document.EvaluatingContext`	A context that evaluates placeholders itself, for an expression language of your choice.
`Page`	A page: `getName`, `getId`, `getModel`, `getDocument`, `getTags`, `createTag`, `remove`.
`Model`	The page's settings: grid, grid size, background color and image, shadow, page view. `getRoot`.
`Root`	The root cell: `getLayers`, `createLayer`.
`Layer`	`getElements`, `createNode`, `createConnection(source, target)`.
`Node`	A shape, and a layer of its own. `getGeometry`, `getConnectionPoints`, `createConnectionPoint`, incoming and outgoing connections with and without connection points, a `NodeStyle`.
`Connection`	`getSource`, `getTarget` and their setters, source and target points, `getPoints`, label `getOffset`, `createNode` for labels, a `ConnectionStyle`.
`ConnectionPoint`, `ConnectionPointSpec`	A position relative to a node, `x` and `y` from 0 to 1, offsets `dx` and `dy` in pixels, and `perimeter`.

Type	What it is for
`Element`	The base of everything: `getURI`, `getMarker`, `getElement` for the DOM, `accept` visitors, `stream`, `getModelElementById`, `getModelElementByProperty`, `getModelElementsByProperty`.
`ModelElement`	The base of the root, layers, nodes and connections: `getId`, `getLabel`, `getTooltip`, `getLink`, `getStyle`, `getProperty`, `setProperty`, `getPropertyNames`, `getTags`, `isVisible`, `getParent`, `getModel`, `getPath`, `getPosition`, `getLinkTarget`, `isTargetLink`, `select`, `resolve`, `resolveURI`, `getEnumarateValue`, `remove`.
`ConnectionBase`	`PARENT`, `SOURCE` or `TARGET`: the logical parent of connections for visitors, streams and URIs.
`Geometry`, `Rectangle`, `Point`, `PointList`	Positions and sizes, the alternate bounds of a collapsed container, and the points of a connection.
`Tag`, `Marker`, `LinkTarget`, `Connectable`, `LayerElement`	As in Chapter 6.

A.2. org.nasdanika.sdk.runtime.drawio.style

Type	Typed methods, on top of `Map<String, String>`
`Style`	`opacity`, `rounded`, `shadow`, `enumerate`, `enumerateValue`
`LineStyle`	`color` (`strokeColor`), `width` (`strokeWidth`), `dashed`
`NodeStyle`	`backgroundColor` (`fillColor`), `shape`, `align`, `verticalAlign`, `fontSize`, `fontColor`, `fontStyle`, `labelBackgroundColor`, `labelBorderColor`, `rotation`, `container`, `collapsible`
`ConnectionStyle`	`edgeStyle`, `startArrow`, `endArrow`, `startFill`, `endFill`, `jump`
`Arrow`, `Jump`	The common values of the arrow and line jump keys.

A.3. org.nasdanika.sdk.runtime.drawio.comparators

Comparator	Order
`LabelModelElementComparator`	By label text, HTML removed, empty labels last.
`PropertyModelElementComparator`	By the value of a property, empty values last.

Comparator	Order
`PositionModelElementComparator`	By position in the parent: the order of the file.
`FlowComparator`	By the number of steps along connections from one element to the other, optionally only along connections that match a predicate. Not total.
`EnumerateComparator`	By Draw.io enumeration values, compared as dotted paths of numbers and words. Not total.
`AngularNodeComparator`	By angle around a hub node, counterclockwise from a base angle in radians, twelve o'clock by default.
`CartesianNodeComparator`	By columns and rows, in one of eight directions.
`Comparators`	Keys for the orders above, such as `label`, `property`, `flow`, `clockwise`, `right-down`, `position`, `enumerate`, and their reverses.

A.4. `org.nasdanika.sdk.runtime.drawio.util`

`Util.getConnectableNode` returns the node of a connection point, or the node itself. `Util.withLinkTargets` wraps a visitor so that it visits link targets; `Util.traverser` wraps one so that it visits connected elements, parents and link targets. `Util.loadStyle` parses a style string into a read-only map. `DelimitedStringMap` and the `AbstractSplitJoin` collections are the live views behind styles and tags, available for attributes of your own in the same formats. `ConfigurationException` is thrown for links and selectors that cannot be resolved, with the marker of the element at fault.



> `Util.traverser` has two overloads. The one that takes a `BiFunction` visits the target of a connection twice and never its source: it calls `getConnectableNode(connection.getTarget())` for both. The `Consumer` overload is correct. The `package-info.java` of the module links to the documentation of Nasdanika core, and the Javadoc of `Document.Context.getVariables` still mentions `$spel` magic properties, which the module no longer has; `getVariables` itself is not used by the module. The `impl` and `style.impl` packages are exported, which makes the implementation classes part of the API.

A.5. org.nasdanika.sdk.runtime.drawio.playwright

In the module drawio-playwright (Chapter 13).

Type	What it is for
Rendering	Renders pages with Draw.io's viewer in a browser: png and svg for a document's first page, a page, or pages with a file name function and an optional PageBody; video for frames; inViewer for full control; fileNames(extension). A record with with methods; DEFAULT.
Rendering.PageBody	What to do with a page once the viewer shows it: run(viewer, page).
Viewer	A document open in the viewer: page, document, show, click by ID or element, png, svg, session. The viewer is the browser's global variable nasdanikaViewer.
Frame	A page of a video, its duration and an optional PageBody.
Pages	Pages to render: byId, byName, all.

A.6. Selectors

Selector	Selects
..	The parent.
child, child[id], child[name=value]	Children, all or filtered.
link-target	What the link points to: an element, or the root of a page.
incoming-connection, outgoing-connection, with [id] or [name=value]	A node's connections, including those of its connection points.
source, target	A connection's ends.
segment/segment/⋯	A path. As a property name, the last segment is the property.

A.7. Links

Link	Example
data:page/id,[<document>#]<page ID>	data:page/id,containers

Link	Example
`data:page/name,[<document>#]<page name>`	`data:page/name,overview.drawio#System+Context`
`data:element/id,[[<document>#]<page>/]<element ID>`	`data:element/id,id,system-context/ibs`
`data:element/name,[[<document>#]<page>/]<label>`	`data:element/name,name,Page+2/Linked`

In element links, `<page>` is `id,<page ID>` or `name,<page name>`. Names and labels are URL-encoded.

Appendix B: Migrating from Nasdanika core

The module is a salvage of `org.nasdanika.core:drawio`, the Draw.io API of Nasdanika core. Code written against core moves to it with a few mechanical changes, unless it uses what was left out.

B.1. What moves

Nasdanika core	The runtime
Maven `org.nasdanika.core:drawio`	Maven `org.nasdanika.sdk.runtime:drawio`, versioned with the runtime's BOM
Java module `org.nasdanika.drawio`	Java module `org.nasdanika.sdk.runtime.drawio`
Package `org.nasdanika.drawio`	Package `org.nasdanika.sdk.runtime.drawio`, with `comparators`, `style` and `impl` subpackages as before
`org.nasdanika.drawio.Util`	`org.nasdanika.sdk.runtime.drawio.util.Util`
`org.nasdanika.common.Util`, `NasdanikaException`	`org.nasdanika.sdk.runtime.common.Util`, `NasdanikaException`, in the runtime's `common` module

`Document`, its `Context`, the element interfaces, links, selectors, property paths, placeholders with `$style`, styles, comparators and PNG loading work as they did.

B.2. What was left out

Graph processing. In core, `Element`, `Node` and `Connection` extend the interfaces of Nasdanika's graph module, so diagrams can be made executable with graph processors and dispatched to `@Handler` methods. In the runtime, the elements extend nothing outside the module (Chapter 14).

The Ecore model of Draw.io documents. Core's `drawio.model` module and `Document.toModelDocument()` are not in this module. The runtime has a Draw.io model of its own, in `models/drawio`, built on its diagram and presentation models, with tags as objects and connections navigable from both ends. A document becomes a model by loading the `.drawio` file, or a PNG or SVG image with the document in it, into a resource set, not by converting a `Document` (Chapter 14). Its classes are named after the diagram model's, `Document`, `Page`, `Layer`, `Node` and `Connection`, not

`DrawioDocument` and the like as in core.

Semantic mapping. Core's mapping module, which maps diagram elements to the objects of Ecore models by their properties, is not in the runtime. The runtime's diagram model has the place for the result, the semantic elements of an element, and mapping by properties is planned (Chapter 14).

Spring Expression Language. The `$spel:` magic property, `data:spel,` links and the SpEL evaluation context are gone, together with the dependency on Spring. `Document.EvaluatingContext` remains as the hook for an expression language: implement `evaluate` to resolve placeholders with the language of your choice.

Layout. `Util.layout` and `Util.forceLayout`, which laid out nodes with JGraphT, are gone with their dependency. In the runtime, layout belongs to the diagram model, with Eclipse ELK, in `models/diagram/elk`, which lays out the pages of the Draw.io model (Chapter 14).

The capability factories that registered Draw.io as a resource type and as processor factories with core's capability loader. The Draw.io model registers the resource type now, with every resource set of the runtime.

B.3. Dependencies

Core's module required `java.desktop`, the graph, mapping, capability and Draw.io model modules, Ecore, and Apache Commons Lang, most of them transitively. The runtime's requires `common` and `java.xml` transitively, and `java.desktop`, jsoup, JSON, Commons Codec and Commons Text without exporting them (Chapter 5). Code that used `java.desktop` or one of these libraries through the Draw.io module has to require it itself.

Appendix C: Building this book, and editing its figures

This book is AsciiDoc in the `drawio` module of the runtime, in `src/docs/asciidoc`. It is built with the module, after the module's tests have passed.

C.1. Building

From the runtime's root directory:

```
mvn -pl drawio -am verify                          # the module, its tests and this book
mvn -pl drawio verify -Dzephyr.editor-notes=false   # without editor's notes, for publication
mvn -pl drawio verify -Dasciidoctor.skip=true       # the module without the book
```

The build writes three editions into `drawio/target/book`, named after the module and its version: a PDF at 189 by 246 mm (7.44 by 9.69 inches), a single HTML page with every image embedded, and an EPUB. The PDF and the HTML are attached to the module's artifact with the classifier `book`, and the module's test sources with the classifier `test-sources`, so the code behind every listing is published with the book.

A module of the runtime has a Nasdanika Zephyr Book when it has `src/docs/asciidoc/index.adoc`. That file activates the `zephyr-book` profile of the runtime's parent POM, which renders and attaches the book; the module's own POM changes only to render the EPUB, which this book has because it is on Leanpub. The theme, the stylesheets and the title page are in the runtime's `doc/zephyr` directory and shared by all the Zephyr Books.

C.2. Conventions

- **Listings are tagged regions of the module's tests**, included with `include::{booktests}/...[tag=name,indent=0]`. Callouts are comments of the form `// <1>` in the test source.
- **A listing line holds 96 characters** at the PDF's code font size. Wrap longer lines in the test source.
- **Warnings fail the build**: a missing include, a missing tag, a broken cross-reference.

- **Editor's notes** are `IMPORTANT` blocks with the caption "Editor's note", inside `ifdef::editor-notes[]`.

C.3. Editing the figures

The figures are in the module's test resources, in `src/test/resources/org/nasdanika/sdk/runtime/drawio/tests/book`, as PNG images with the diagram embedded. To change one, open the PNG in Draw.io: the desktop application or the Visual Studio Code extension open it directly, and the web application opens it with *File*, *Open from*, *Device*. Edit the diagram, then save it as PNG again with *Include a copy of my diagram* checked, a zoom of 200 percent, and *Transparent Background* unchecked.

The figures whose names start with `rendering-` are outputs instead: the tests of the `drawio-playwright` module render them into `drawio-playwright/target/rendering`, from which they were copied here. Render them again with `mvn -pl drawio-playwright -am verify` after changing the diagram they show, `banking.drawio` in that module's test resources.

The other figures are test inputs, so a change to one can fail a test: rename an element and the listing that finds it by ID fails, change an arrow of the metamodel figure and the test that checks it against the API fails. That is the point. Run the tests after editing a figure, and change the test, or the figure, until they agree.



> The figures of this edition were authored as XML and rendered with Draw.io's viewer, version 32.5.5, in headless Chrome at 2x, then written as PNG with the diagram in a `tEXt` chunk named `mxfile`, which is what Draw.io writes. Check that each opens and saves in the desktop application before the first release, and resave them from Draw.io if its rendering differs.

Appendix D: Glossary

Connection

A line between two connectables. Either end may be missing.

Connection base

Which element a connection is a logical child of: its parent in the file, its source or its target.

Connection point

A fixed point on the outline of a node where connections leave or arrive.

Consumption cost

The effort readers spend to get what they need from a diagram.

Context

What a document is loaded with: the streams of linked documents, properties for placeholders, and a filter for properties.

Custom action

A step that Draw.io's viewer runs when a shape with a `data:action/json` link is clicked: show, hide, fade, highlight, restyle or animate cells.

Diagram as data

A diagram whose elements carry properties with agreed meanings that programs read.

Editable image

A PNG or SVG image that carries the diagram it shows, so that Draw.io opens it for editing and the API loads it.

Element link

A link from a diagram element to another element, on the same page, another page or another document, in the form `data:element/⋯`.

Entity

A thing a diagram element stands for.

Frame

A page of a rendered video, shown for a time, with an optional function that runs once the page is shown.

Layer

A part of a page that is shown and hidden as a whole. Every page has at least one.

Node

A shape. A node can contain nodes and connections.

Placeholder

A `%name%` in a label or a property, replaced with the value of the property `name` when the element has placeholders on.

Production cost

The effort of drawing a diagram and keeping it current.

Property

A named value of a diagram element, set in Draw.io's *Edit Data* dialog.

Property path

A property name with a selector in front of it, such as `outgoing-connection/target/owner`, which reads a property of a related element.

Representation filtering

Changing how a diagram looks, from code, to show the current state of what it describes, without changing what people drew.

Selector

A path of segments, such as `..`, `child` and `outgoing-connection`, which selects elements relative to an element.

Tag

A name that elements of a page share, so that they can be shown and hidden together.

Viewer

Draw.io's script that shows diagrams in web pages, with pages, layers, links and custom actions. The module `drawio-playwright` renders diagrams with it in a browser.

Visual communication continuum

The range of structure a diagram can carry, from a picture to a diagram mapped to a model, with each step adding a little to the same file.